# BUILD ONCE.
# CONFIGURE CLIENTS.

72 development prompts
in the order to use them

### Your stack

MySQL, Next.js/React, Node.js and Python.

### Your platform

Client dashboard, connected numbers, no-code rules and workflows.

### Your delivery

Reusable profiles, verified onboarding and portable clients.

Omnichannel leads + AI chat + approved AI calling
Redis + Qdrant search + private storage

Copy one prompt, implement one stage,
review its evidence, then continue.

Proposed design / 9 October 2026

# How to use this promptbook

Start with Prompt 00 on the next page, then Prompt 01. Give the coding agent one numbered prompt at a time. Each prompt declares the goal, prerequisites, files, data/API/configuration changes, security, tests, outputs, completion and rollback.

### 1. Save the durable context

Keep this promptbook, docs/architecture.md, the stage ledger and the latest stage report with the repository. The Markdown companion contains the same complete prompts for easy copying.

### 2. Run one stage and review the evidence

The agent inspects current work, implements only the requested stage, runs checks and updates docs/stages/NN.md. Fix relevant failed prerequisites before continuing.

### 3. Continue in order

Paste the next prompt with the last result. In a new chat, paste Prompt 00 again and supply the architecture/ledger so the agent does not silently redesign the system.

### 4. Handle external blockers honestly

A technically tested capability can remain pending_external. Continue unrelated work with fake adapters, but keep the blocked live capability off and never report its live checks as passed.

> **Build the platform once; onboard clients afterwards**
>
> These prompts develop the reusable product. Supported client delivery then uses the dashboard and configuration packages. Adding a completely new vendor/action still needs reviewed development.

> **A stable foundation, with controlled upgrades**
>
> No promptbook can guarantee permanent perfection. Versioned contracts, explicit migrations and acceptance checks reduce disruptive rewrites. One-to-three-day client delivery depends on ready requirements, assets, credentials and external approvals.

Prompt pages: 10-45, two complete prompts per page. Index: 8-9. Sources: 46-48. This promptbook is the new stack/configuration authority; the earlier 45-page PDF remains conceptual background where compatible.

# Prompt 00: master implementation policy

Act as the implementation engineer for this approved multi-client omnichannel lead platform. Read docs/architecture.md, repository instructions and the stage ledger before every stage. This promptbook replaces the earlier PostgreSQL/pgvector and own-business assumptions with MySQL/Qdrant and multi-client onboarding. Preserve the three systems: lead ingestion/routing, AI chat with RAG/memory, and approved AI calling.

Use Next.js with React/TypeScript for one dashboard, Node/Fastify for business APIs, Node/BullMQ workers, MySQL/InnoDB as durable authority, Python/FastAPI for bounded AI/retrieval computation, Redis for queues/cache/rate limits and Qdrant for derived vector search. Do not silently replace these choices. Inspect and extend an existing working repository; scaffold if empty. Verify and pin compatible supported dependencies from primary documentation.

Build one shared product with tenant configuration, not client-specific code forks. Supported client settings, assets, numbers, rules, forms, AI profiles, knowledge and workflows must be editable in the dashboard. Node owns business writes and external effects. Python cannot directly write business data or send/dial. All tenant access, worker jobs, search, tools, media and exports require trusted tenant/actor scope. MySQL does not supply the PostgreSQL RLS assumed by the earlier design; implement scoped repositories and tenant-safe constraints.

Commit verified inbox events before acknowledgement and business changes plus outbox in one transaction. Consume at least once with idempotency/effect receipts. Reconcile ambiguous external sends/dials before retry. Keep deterministic routing and one active assignment. Recheck current consent, suppression, channel policy, permissions and human ownership before execution. AI cannot grant consent, approve credit, choose providers, grant roles or execute arbitrary code/SQL/HTTP. Private memory/status never enters shared semantic cache.

Implement only the numbered stage requested. Apply its declared compatible migrations; preserve existing secrets/data. If an unexpected change breaks architecture/contracts/security, explain change, reason, impact, migration and rollback, then wait for approval. Verify current Meta/provider documentation and actual account access; label blocked/unverified details. Keep unapproved capabilities disabled. Use synthetic data only when labelled.

Run preflight, meaningful stage tests and postflight. Return actual evidence and update docs/stages/NN.md and the stage ledger. Do not fabricate links, metrics, approvals or completion. Stop for relevant failed prerequisites; unrelated work may proceed while external approval is recorded as pending and the affected capability remains off. Do not implement the next stage until asked.

# The architecture you will implement

| Layer | Decision |
|---|---|
| Dashboard | One Next.js/React TypeScript app: platform admin, client admin, operators and providers. |
| Business core | Node.js/Fastify modular API plus Node worker. Zod contracts; Prisma/MySQL with explicit locking where needed. |
| Durable data | MySQL 8.4 LTS/InnoDB owns tenants, leads, consent, configs, workflow state, inbox/outbox and receipts. |
| Temporary state | Redis/BullMQ queues, rate limits, hot context and cache. Durable jobs recover from MySQL. |
| AI computation | Private Python/FastAPI for model calls, parsing, embeddings and retrieval; no direct business writes/sends. |
| Vector search | Qdrant, preferably managed for the small team, with trusted tenant/ACL filters and versioned collections. |
| Files and external routes | Private object storage; verified Meta adapters; approved Exotel/managed ElevenAgents voice route. |

React is used inside Next.js. Python handles AI work while Node retains policy, transactions and effects. The business core remains modular instead of splitting every feature into a separate microservice.

> **Why Qdrant here**
>
> Recommended fit: filtered vector search and a documented distributed deployment path. Qdrant supports payload tenancy and optional tenant-focused placement; authorization must still be enforced. This is not a claim that one database is universally the most advanced. Confirm features in the pinned version. [S04-S05, S23]

Superseded choices: PostgreSQL/pgvector -> MySQL/Qdrant; own-business-only setup -> verified multi-client connector onboarding. No existing data migration is claimed or executed by this document.

# No-code configuration and data boundaries

| Dashboard settings | Stored/published contract |
|---|---|
| Client + branding | Business profile, locale/timezone, supported products, roles and approved capability package. |
| Forms + lead fields | Versioned field definitions, validation and source mapping; stable IDs preserve older records. |
| Providers + routing | Eligibility, geography, priority, capacity, response timeout and deterministic fallback. |
| Numbers + channels | Verified asset ownership, connector instance, credential reference, registration/permissions and default sender. |
| AI + knowledge | Approved agent sections/models/languages, knowledge versions, memory policy and bounded budgets. |
| Workflows + schedules | Typed event/condition/action/wait/handoff nodes with simulation, publication and immutable revisions. |
| Privacy + cost | Approved consent/retention policies, quotas, usage rates and restricted export/transfer decisions. |

> **Publish safely**
> Draft -> validate -> simulate -> review diff -> publish -> observe -> restore prior valid revision. Running workflows pin a definition, while current withdrawal, authorization and account policy always win.

MySQL data is tenant-scoped through repository enforcement and same-tenant constraints. Qdrant is a derived index: filter and verify ownership/version. Redis keys, jobs, files, tools, exports and live streams carry trusted scope. Customer prompts cannot override platform safety.

Internal API/file names in the stages are proposed implementation contracts, not external URLs or deployed endpoints. Adapt paths to an existing repository rather than duplicating working modules.

# The reusable client delivery process

## A supported new client

**1. Create a workspace**

Select package/profile; invite admin; choose branding, fields, products and language.

↓

**2. Configure requirements**

Set providers/routing, knowledge, agent questions, workflows, schedules and budgets.

↓

**3. Connect owned assets**

Use authorized channel onboarding; add verified numbers and vault-held credential references.

↓

**4. Simulate and verify**

No-effect sandbox first; then approved account/recipient tests, privacy and isolation checks.

↓

**5. Activate and hand over**

Enable only ready capabilities, record owner/evidence and show recovery/support procedures.

### Changing a phone number is a workflow

A new sender needs ownership, registration/verification, access and tests. Dashboard editing cannot replace Meta/provider verification. Existing queued intents retain their sender or are explicitly cancelled/recreated.

### Portable delivery

Shared SaaS or a dedicated deployment uses the same product release and configuration. Transfers export only authorized client data/configuration; external assets and secrets must be re-authorized, with an explicit cutover that avoids duplicate contact.

# Verification and recovery prompts

## Standard preflight

Read the architecture, configuration/API/event schemas and previous stage evidence. Inspect actual files, migrations, lockfiles and working changes. List the requested stage's dependencies and resolve any relevant failures. Run the appropriate prior tests; note blocked external approvals explicitly. State the intended file/data/contract changes and verify they match the stage. Protect secrets and existing working configuration. Unexpected breaking deviations need an explained, reviewable change and approval.

## Standard post-implementation check

Run the stage's required unit/integration/contract/browser checks and applicable tenant, duplicate, policy and recovery cases. Inspect the resulting UI and durable records, not just mocks. Record exact checks executed, real results, changed files, schema/config/event versions, operator steps, remaining blockers and rollback. Update the stage ledger as verified, technically verified/pending external, or failed. Never count unrun/live-blocked checks as passed. Proceed only when the next stage's dependencies are proven.

## Paste this when a stage fails

> **Repair before the next stage**
>
> Do not implement the next stage yet. Read the master policy, current architecture and the last stage report. Reproduce the reported failure, inspect contracts and identify the smallest compliant fix. Preserve tenant isolation, idempotency, consent, effect reconciliation and secrets. If a breaking change is necessary, explain the impact and migration and wait for approval. Otherwise fix the defect, rerun relevant tests plus regressions, update the evidence and show why the stage is now safe to continue.

Use verified, technically verified/pending external, and failed as distinct ledger states. Synthetic fixtures are always labelled. Passing mocks prove internal behaviour; approved account tests are separately required for live activation.

Application/database/API/event semantics may evolve only through declared stage work or an explained change with migration, compatibility and rollback. Never silently replace this stack, routing or tenant model.

# Ordered prompts 01-36

Go in numerical order. Each stage builds on the prior evidence and the explicit dependencies in its prompt. PDF bookmarks link each individual prompt.


| | |
|---|---|
| **01** Inspect and freeze the new architecture (p. 10) | **19** Create reusable client profiles and white-label settings (p. 19) |
| **02** Create the repository and local environment (p. 10) | **20** Build dynamic forms and custom-field mapping (p. 19) |
| **03** Establish quality checks and stage evidence (p. 11) | **21** Build the no-code routing rule editor (p. 20) |
| **04** Define versioned API, event and configuration contracts (p. 11) | **22** Define the no-code workflow language (p. 20) |
| **05** Implement MySQL access and migration discipline (p. 12) | **23** Implement durable workflow execution (p. 21) |
| **06** Implement authentication and tenant membership (p. 12) | **24** Build workflow builder, simulator and publishing UI (p. 21) |
| **07** Create permissions and support-access controls (p. 13) | **25** Create connector registry and capability lifecycle (p. 22) |
| **08** Implement secrets and per-client credential references (p. 13) | **26** Implement the communication policy and intent service (p. 22) |
| **09** Build tenant lifecycle and provisioning (p. 14) | **27** Create multi-client Meta readiness and review checklist (p. 23) |
| **10** Create contact, identity and enquiry records (p. 14) | **28** Implement secure customer authorization and Embedded Signup (p. 23) |
| **11** Implement consent, preferences and suppression (p. 15) | **29** Build phone-number onboarding and safe replacement (p. 24) |
| **12** Build providers and assignment invariants (p. 15) | **30** Implement verified shared Meta webhook delivery (p. 24) |
| **13** Build the unified conversation store (p. 16) | **31** Implement Facebook Lead Ads retrieval adapter (p. 25) |
| **14** Implement durable webhook inbox and event outbox (p. 16) | **32** Implement normalization, qualification fields and golden routing (p. 25) |
| **15** Configure Redis, BullMQ and fair worker scheduling (p. 17) | **33** Implement WhatsApp outbound intents and status reconciliation (p. 26) |
| **16** Create audit, idempotency and export primitives (p. 17) | **34** Implement WhatsApp inbound conversation processing (p. 26) |
| **17** Build platform-admin and client dashboard shells (p. 18) | **35** Build WhatsApp template, media and account-health management (p. 27) |
| **18** Implement versioned configuration and publishing (p. 18) | **36** Implement Instagram through its verified login path (p. 27) |


Use Prompt 00 first. Source IDs on stage cards refer to the official directory on pages 46-48.

# Ordered prompts 37-72

Go in numerical order. Each stage builds on the prior evidence and the explicit dependencies in its prompt. PDF bookmarks link each individual prompt.


| | |
|---|---|
| **37** Implement Messenger with scoped Page access (p. 28) | **55** Add reusable outbound integrations and secure mappings (p. 37) |
| **38** Add website forms and controlled contact import (p. 28) | **56** Create delivery packages and guided client onboarding (p. 37) |
| **39** Build the human inbox and operator handoff (p. 29) | **57** Implement data classification, retention and deletion (p. 38) |
| **40** Build lead-management and provider workspaces (p. 29) | **58** Harden the dashboard, APIs, uploads and integrations (p. 38) |
| **41** Implement the private Python AI service boundary (p. 30) | **59** Implement per-client usage, quotas and budget limits (p. 39) |
| **42** Build knowledge upload, review and lifecycle (p. 30) | **60** Build operational and client analytics (p. 39) |
| **43** Implement Qdrant collections and index consistency (p. 31) | **61** Implement monitoring, alerts and recovery console (p. 40) |
| **44** Implement scoped RAG retrieval and evidence (p. 31) | **62** Package a portable deployment and infrastructure configuration (p. 40) |
| **45** Implement structured memory, summaries and archive (p. 32) | **63** Implement backups, restore and disaster-recovery drill (p. 41) |
| **46** Implement public semantic cache using Redis and Qdrant (p. 32) | **64** Benchmark and plan MySQL/Qdrant growth (p. 41) |
| **47** Create authorized AI tools and runtime prompt composition (p. 33) | **65** Create AI evaluation and regression gates (p. 42) |
| **48** Implement omnichannel AI chat and human fallback (p. 33) | **66** Run official channel contract and account acceptance tests (p. 42) |
| **49** Create India voice readiness and compliance workflow (p. 34) | **67** Run failure, race and load verification (p. 43) |
| **50** Implement Exotel voice connectivity adapter (p. 34) | **68** Verify complete business journeys and dashboard usability (p. 43) |
| **51** Configure managed AI voice agents and bounded tools (p. 35) | **69** Perform release security, privacy and compliance review (p. 44) |
| **52** Implement post-call outcomes and callback reconciliation (p. 35) | **70** Implement tenant export, transfer and dedicated import (p. 44) |
| **53** Implement qualification and explainable lead scoring (p. 36) | **71** Deploy the approved pilot with rollback evidence (p. 45) |
| **54** Build scheduling, campaigns and approved follow-up automation (p. 36) | **72** Onboard a second client with zero code changes (p. 45) |


Use Prompt 00 first. Source IDs on stage cards refer to the official directory on pages 46-48.

# Prompts 01-02

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 01 | Inspect and freeze the new architecture

**OBJECTIVE:** Establish the MySQL-based multi-client architecture before coding the product. **PREREQUISITES:** Approved master policy and an inspected empty/existing repository.

**FILES:** docs/architecture.md; docs/decisions/; docs/stage-ledger.md; repository AGENTS.md. **DATABASE:** Inventory existing storage; no schema or data changes.

**API CONTRACTS:** Document proposed /api/v1, event v1 and private AI-service contracts. **ENV / CONFIG:** Inventory names only; never print existing values.

**IMPLEMENT:** Inspect code, lockfiles, deployment and current integrations. For an empty repository, propose the exact scaffold. Adopt Next.js/React, TypeScript Node/Fastify, MySQL/InnoDB, Redis/BullMQ, Python/FastAPI and Qdrant. Document the move from PostgreSQL/pgvector to MySQL/Qdrant and own-business to multi-client onboarding. Record owners, scope, MVP/full-platform split and dependency versions from official sources. No production feature implementation in this stage.

**SECURITY:** Preserve existing secrets/configuration; define mandatory tenant isolation and disabled-by-default calling.

**TESTS:** Validate a requirements-to-stage coverage matrix and architecture boundaries. Run existing checks, if present, and report unrun checks honestly.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/01.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Documentation-only revert; preserve inspection evidence. **PRESERVE:** No silent stack, contract, security or routing changes.

PRIMARY REFERENCES: S01 S02 S03 S04

### PROMPT 02 | Create the repository and local environment

**OBJECTIVE:** Create one maintainable workspace with explicit Node and Python boundaries. **PREREQUISITES:** Prompt 01 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** apps/web; apps/api; apps/worker; services/ai-service; packages/contracts; packages/db; deploy/local. **DATABASE:** Provision an isolated local MySQL database, not a production database.

**API CONTRACTS:** Health/readiness contracts only; internal AI routes are private. **ENV / CONFIG:** DATABASE_URL, REDIS_URL, QDRANT_URL, AI_SERVICE_URL, APP_ORIGIN, NODE_ENV.

**IMPLEMENT:** Scaffold only missing components. Next.js owns the React dashboard; Fastify owns business APIs; Node worker owns durable effects; Python supplies AI/retrieval computation. Use pnpm, supported Node LTS, Python virtual environment, lockfiles and local containers for MySQL, Redis and Qdrant. Separate examples from actual secret files and document startup/shutdown.

**SECURITY:** Bind local dependencies privately; exclude .env, recordings, tokens and tenant exports from Git.

**TESTS:** Clean-checkout startup, health checks, dependency connections and a graceful shutdown. Prove Python has no MySQL write credentials.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/02.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Stop local containers without deleting volumes; revert scaffold changes. **PRESERVE:** One shared codebase; no per-client code forks.

PRIMARY REFERENCES: S01 S02 S03 S06

# Prompts 03-04

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 03 | Establish quality checks and stage evidence

**OBJECTIVE:** Make every later stage verifiable in a clean environment. **PREREQUISITES:** Prompt 02 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** CI config; packages/testing; tests/; docs/preflight.md; docs/postflight.md. **DATABASE:** Isolated ephemeral MySQL test database and explicitly synthetic tenant fixtures.

**API CONTRACTS:** Contract-test harness for API, events and Python request/response. **ENV / CONFIG:** TEST_DATABASE_URL; test-only credentials; no real service tokens.

**IMPLEMENT:** Configure TypeScript checks, linting, meaningful Node/Python unit and integration tests and Playwright browser tests. Persist stage status and evidence links. Preflight inspects changes/contracts and reruns relevant prior gates. Postflight records executed checks, residual risks, migrations and rollback. Seed two unrelated synthetic tenants for leakage tests.

**SECURITY:** CI redacts secrets; fixtures cannot message real recipients; outbound adapters are fake in tests.

**TESTS:** Intentionally failing validation fails CI; the cross-tenant fixture cannot access another tenant; a clean build is reproducible.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/03.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Revert check configuration while retaining stage evidence. **PRESERVE:** No fabricated coverage, timings or completion claims.

### PROMPT 04 | Define versioned API, event and configuration contracts

**OBJECTIVE:** Freeze interfaces before features depend on them. **PREREQUISITES:** Prompt 03 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** packages/contracts; docs/contracts; OpenAPI generation; Python schema generation. **DATABASE:** Specify IDs, UTC timestamps, version columns and durable receipt keys.

**API CONTRACTS:** /api/v1; private /internal/v1 AI schema; event envelope v1. **ENV / CONFIG:** API_VERSION; AI_SERVICE_AUDIENCE; validated global schema defaults.

**IMPLEMENT:** Use Zod/JSON Schema and compatible Python models. Define tenant/actor context, structured errors, pagination, correlation, idempotency, optimistic concurrency and async job status. Event fields include id/type/schema_version/tenant/aggregate/version/time/correlation/causation. Document configuration versioning and compatibility. Generate clients from one contract source instead of manually duplicating types.

**SECURITY:** Caller-supplied tenant IDs cannot authorize access; internal service responses remain untrusted input.

**TESTS:** Invalid payloads rejected; generated Node/Python clients agree; stale versions produce a documented conflict; compatible event fixtures round-trip.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/04.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Keep prior contract readers and revert new writers. **PRESERVE:** No incompatible contract edits without migration and approval.

PRIMARY REFERENCES: S07

# Prompts 05-06

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 05 | Implement MySQL access and migration discipline

**OBJECTIVE:** Make the Node core the sole owner of durable business writes. **PREREQUISITES:** Prompt 04 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** packages/db/schema; migrations; scoped repositories; apps/api database plugin. **DATABASE:** MySQL 8.4 LTS/InnoDB, utf8mb4, UTC conventions; tenant composite keys/indexes.

**API CONTRACTS:** Repository transactions and unit-of-work interface; no public route yet. **ENV / CONFIG:** DATABASE_URL, DB_POOL_LIMIT, DB_QUERY_TIMEOUT; migration role secret reference.

**IMPLEMENT:** Select and pin a verified Prisma/MySQL combination. Define a transaction wrapper, bounded pool/timeouts, migration ledger and safe raw SQL escape hatch for locking. Require tenant context in every tenant-owned repository call. Use composite foreign keys to enforce same-tenant relationships; MySQL tenancy is explicit application enforcement, not assumed PostgreSQL RLS.

**SECURITY:** Least-privilege runtime account; separate migration role; parameterized queries; prohibit unscoped generic repository methods.

**TESTS:** Migration on a fresh database and upgraded fixture; invalid cross-tenant foreign key rejected; rollback/deadlock handling tested.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/05.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Use a compatible forward fix or restore approved test snapshot; do not drop production data. **PRESERVE:** Database is MySQL; Qdrant is a derived search index.

PRIMARY REFERENCES: S01 S08 S09

### PROMPT 06 | Implement authentication and tenant membership

**OBJECTIVE:** Provide trusted identities without inventing a custom authentication protocol. **PREREQUISITES:** Prompt 05 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** apps/web/auth; apps/api/auth; packages/db membership models. **DATABASE:** Users, tenants, memberships, invitations, sessions/audit references.

**API CONTRACTS:** /auth/session; /tenants; /tenants/:id/switch with membership checks. **ENV / CONFIG:** OIDC_ISSUER, OIDC_CLIENT_ID, OIDC_CLIENT_SECRET_REF, SESSION_SECRET_REF.

**IMPLEMENT:** Use a vetted OIDC/session integration and document its verified configuration. Enforce platform owner, tenant admin, operator, provider and auditor roles. Make tenant switching explicit; authenticate server-side calls; revalidate memberships on sensitive operations. Implement invitation expiry, account disable and required MFA policy for privileged roles.

**SECURITY:** Secure HttpOnly cookies, CSRF protection for cookie writes, session fixation prevention and server-enforced authorization.

**TESTS:** Expired invitation/session blocked; removed membership loses access; forged tenant/header and provider role escalation rejected.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/06.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable new login path behind a flag; preserve identity and membership evidence. **PRESERVE:** Authentication does not permit cross-tenant data access.

# Prompts 07-08

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 07 | Create permissions and support-access controls

**OBJECTIVE:** Centralize authorization for UI, API, jobs and AI tools. **PREREQUISITES:** Prompt 06 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** packages/security; permission matrix; support-access UI; audit integration. **DATABASE:** Role permissions and time-limited support grants with reason/expiry.

**API CONTRACTS:** /support-grants; reusable authorize(actor, tenant, action, resource). **ENV / CONFIG:** SUPPORT_ACCESS_MAX_DURATION; privileged operation feature flags.

**IMPLEMENT:** Define action/resource permissions, provider assignment scope and delegated tenant admin rights. Add explicit, audited platform-support access with expiry and revocation; avoid silent impersonation. Workers use service principals with tenant scope, not a universal admin shortcut. Hide prohibited controls while always enforcing server permissions.

**SECURITY:** Fail closed for missing tenant/actor; no AI, workflow or configuration can grant roles.

**TESTS:** All role/resource combinations tested; expired support grant fails; unauthorized jobs and tools are rejected.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/07.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable support access; revoke active grants while keeping audit records. **PRESERVE:** No unaudited superuser path or customer-role escalation.

### PROMPT 08 | Implement secrets and per-client credential references

**OBJECTIVE:** Support many clients without tenant tokens in environment files. **PREREQUISITES:** Prompt 07 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** packages/secrets; connector credential service; credential status UI. **DATABASE:** Secret references, credential metadata, owner/scope/expiry; never plaintext credentials.

**API CONTRACTS:** /credentials metadata; restricted create/rotate/revoke operations. **ENV / CONFIG:** SECRET_STORE_URL, SECRET_STORE_AUTH_REF, ENCRYPTION_KEY_REF.

**IMPLEMENT:** Integrate a deployable secret-store adapter. Separate platform app credentials from client-authorized channel/provider credentials. Store references and scope metadata in MySQL; resolve credentials only inside authorized server adapters. Provide masked entry, health verification, rotation and revocation workflows. New clients must not require editing .env or redeploying.

**SECURITY:** Never return raw tokens to browsers, exports, logs or AI context; audit access and support a key-rotation plan.

**TESTS:** Wrong-tenant credential cannot resolve; old credential stops after rotation; failed verification remains inactive; log scan has no secrets.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/08.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Revoke newly issued credentials and restore the prior authorized reference if still valid. **PRESERVE:** No shared unscoped client credential fallback.

# Prompts 09-10

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 09 | Build tenant lifecycle and provisioning

**OBJECTIVE:** Create a recoverable client workspace from the dashboard. **PREREQUISITES:** Prompt 08 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** tenant module; onboarding UI; provisioning worker. **DATABASE:** Tenants, provisioning steps, state transitions, audit and idempotency receipts.

**API CONTRACTS:** POST /tenants; /tenants/:id/provision; /onboarding/status. **ENV / CONFIG:** Default locale/timezone and validated plan identifiers; credentials stay in vault.

**IMPLEMENT:** Implement draft, configured, pending_external, ready, active, paused and offboarding states. Provision internal defaults idempotently with tracked step status. A failed external dependency must not duplicate the tenant on retry. Create the tenant administrator and show exactly what blocks activation. Use a reusable profile, not a copied application.

**SECURITY:** Platform-admin-only creation; no customer can assign their plan or activate an unapproved route.

**TESTS:** Retry provisioning creates one workspace; failure resumes at the correct step; pause blocks new outbound work.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/09.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause tenant and undo unused internal provisioning steps; preserve evidence and data.
**PRESERVE:** Provisioning cannot bypass external ownership or review.

### PROMPT 10 | Create contact, identity and enquiry records

**OBJECTIVE:** Store a person separately from each business enquiry. **PREREQUISITES:** Prompt 09 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** lead/contact modules; packages/db domain models; lead forms. **DATABASE:** Contacts, scoped identities, leads, lead_sources, lead_events and custom-field definitions.

**API CONTRACTS:** /contacts; /leads CRUD with expectedVersion and idempotency keys. **ENV / CONFIG:** IDENTITY_HMAC_KEY_REF; configurable phone-country defaults.

**IMPLEMENT:** Preserve original identity evidence and normalized forms. Scope channel identities to tenant/channel/business asset. Allow several enquiries per contact; match a repeated source lead once. Define status transitions, provenance and structured product/location/finance requirements without claiming credit eligibility. Custom fields extend validated schemas rather than create uncontrolled SQL columns.

**SECURITY:** Encrypt restricted contact values; no merge by name or an unverified phone; mask fields by role.

**TESTS:** Repeated source ID is one enquiry; separate requirements remain separate; similar names do not merge; stale writes conflict.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/10.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Revert UI/feature flag; correct data through audited events, not destructive bulk merges.
**PRESERVE:** Identity linking requires verified evidence.

# Prompts 11-12

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

## PROMPT 11 | Implement consent, preferences and suppression

**OBJECTIVE:** Make contact permission a durable, purpose-specific decision. **PREREQUISITES:** Prompt 10 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** consent module; preferences UI; suppression evaluator. **DATABASE:** Consent evidence, purposes/channels, withdrawals, suppression and policy versions.

**API CONTRACTS:** /consents; /preferences; internal evaluateContactPermission. **ENV / CONFIG:** Approved consent-text version references; no permissive fallback.

**IMPLEMENT:** Record who/what/when/source and applicable notice. Separate service enquiry, marketing, voice and recording choices. Withdrawal immediately suppresses future intents and queues cancellation. Store the reason/evidence for permission decisions and re-evaluate immediately before execution. Legal/business owners supply the actual policies.

**SECURITY:** Operators and AI cannot fabricate consent; deny by default when evidence or policy is missing.

**TESTS:** Opt-out during a queued send/call blocks execution; channel-specific permission does not authorize another channel; withdrawal is auditable.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/11.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable affected automation; never undo a withdrawal to restore delivery. **PRESERVE:** Consent and suppression remain current, not pinned to old workflow versions.

PRIMARY REFERENCES: S10 S11

## PROMPT 12 | Build providers and assignment invariants

**OBJECTIVE:** Protect provider capacity and lead ownership under concurrency. **PREREQUISITES:** Prompt 11 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** provider/assignment modules; database locking adapter. **DATABASE:** Providers, eligibility/rules, capacity counters, assignments and assignment history.

**API CONTRACTS:** /providers; /assignments; accept/reject transitions. **ENV / CONFIG:** Assignment timeout defaults in approved tenant config.

**IMPLEMENT:** Enforce one active assignment using an active assignment slot keyed by tenant/lead, since MySQL lacks PostgreSQL partial-index semantics. Lock the lead/capacity rows consistently, validate capacity and append history atomically. Handle deadlocks with bounded retry and keep stable assignment generations. Scope provider reads to authorized assignments.

**SECURITY:** Provider cannot see other providers or tenants; assignment decisions cannot be overwritten by AI.

**TESTS:** Parallel attempts yield one active assignment; capacity cannot oversubscribe; rejected/expired transitions preserve history.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/12.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause reassignment; repair through audited state transitions and capacity reconciliation. **PRESERVE:** One active assignment and deterministic routing.

PRIMARY REFERENCES: S08 S09

# Prompts 13-14

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 13 | Build the unified conversation store

**OBJECTIVE:** Unify chat history while preserving channel and identity scope. **PREREQUISITES:** Prompt 12 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** conversation module; message schemas; conversation UI API. **DATABASE:** Conversations, messages, message_events, channel identities and human ownership.

**API CONTRACTS:** /conversations; /conversations/:id/messages with cursor pagination. **ENV / CONFIG:** Message/media size limits; attachment storage configuration.

**IMPLEMENT:** Store direction, provider ID, receipt/source timestamps and explicit delivery state. Separate provider events from message content; deduplicate by scoped provider ID. Link enquiries only after authorized identity evidence. Record human takeover and automation pause as state, not UI-only flags. Prepare retention/deletion references.

**SECURITY:** Private conversations never become globally searchable; protect media access and redact restricted content in logs.

**TESTS:** Duplicate message delivery has one effect; old status cannot regress newer status; human takeover prevents automated reply.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/13.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable conversation ingestion for the affected asset; retain recorded messages. **PRESERVE:** No unverified cross-channel merge.

### PROMPT 14 | Implement durable webhook inbox and event outbox

**OBJECTIVE:** Ensure acknowledgements and committed changes survive queue outages. **PREREQUISITES:** Prompt 13 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** inbox/outbox modules; dispatcher; worker receipts; reconciliation job. **DATABASE:** Webhook events/deliveries, outbox_events, consumer_receipts and dispatch leases.

**API CONTRACTS:** Durable acceptWebhook; event v1 dispatcher contract. **ENV / CONFIG:** Dispatch interval, lease duration and bounded retry config.

**IMPLEMENT:** Commit verified inbound events before acknowledgement. Commit business changes and outbox events in the same MySQL transaction. Dispatch at least once, use consumer/effect receipts, and reconcile stale leases. Document ordering by aggregate version and durable dead-letter state. Redis delivery is a wake-up mechanism, not the source of truth.

**SECURITY:** Resolve tenant from verified asset ownership; payload tenant fields cannot override it.

**TESTS:** Crash after commit/before enqueue recovers; replay has one business effect; uncommitted event is not acknowledged.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/14.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause dispatcher; drain/reconcile from MySQL before restarting. **PRESERVE:** No claimed exactly-once guarantee for external providers.

# Prompts 15-16

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 15 | Configure Redis, BullMQ and fair worker scheduling

**OBJECTIVE:** Use Redis for temporary state with per-client fairness. **PREREQUISITES:** Prompt 14 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** apps/worker queues; packages/cache; scheduler; local/production Redis config. **DATABASE:** Durable job/intent status remains in MySQL; Redis stores references and expiring state.

**API CONTRACTS:** Queue job schema v1 with tenant/correlation/event references. **ENV / CONFIG:** REDIS_URL, REDIS_TLS, WORKER_CONCURRENCY; queue retention/rate-limit config.

**IMPLEMENT:** Separate ingestion, communications, AI and deletion queues. Add bounded retry/backoff, per-tenant concurrency/rate budgets and queue-age metrics. Use appropriate Redis persistence and verified BullMQ production settings. Reconcile after Redis loss from the inbox/outbox; locks supplement, never replace, database constraints.

**SECURITY:** No tokens/raw sensitive payloads in jobs; prevent a busy tenant starving others; no public Redis access.

**TESTS:** Redis restart/rebuild preserves durable intents; poison job dead-letters; two tenants progress under noisy-neighbour load.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/15.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Stop workers and reconcile durable pending work; rebuild temporary queues safely. **PRESERVE:** MySQL remains durable authority.

PRIMARY REFERENCES: S12

### PROMPT 16 | Create audit, idempotency and export primitives

**OBJECTIVE:** Track actions and prevent duplicate API effects consistently. **PREREQUISITES:** Prompt 15 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** audit module; idempotency middleware; safe export helpers. **DATABASE:** Audit events and API idempotency receipts with scope, request digest and response reference.

**API CONTRACTS:** Idempotency-Key for writes; /audit scoped cursor reads. **ENV / CONFIG:** Audit redaction rules and retention-policy references.

**IMPLEMENT:** Bind receipts to tenant/actor/operation and request digest; return prior result for exact retries and reject key reuse with a different payload. Log actor, action, version, reason and correlation. Distinguish business audit from debug logs. Create an export manifest/checksum helper used later for tenant transfer.

**SECURITY:** Mask PII; exclude credentials; restrict audit reads and support access; immutable correction history.

**TESTS:** Network retry repeats no effect; changed-payload key fails; unauthorized audit/export access blocked; log scan is clean.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/16.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable new endpoints; preserve receipts and audit necessary for safe replay. **PRESERVE:** Do not erase audit to hide errors.

# Prompts 17-18

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 17 | Build platform-admin and client dashboard shells

**OBJECTIVE:** Make the whole platform operable without source-code changes. **PREREQUISITES:** Prompt 16 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** apps/web layouts/navigation; dashboard pages; shared UI components. **DATABASE:** User preferences only; consume scoped existing records.

**API CONTRACTS:** Dashboard summary/read APIs; trusted server-side tenant switching. **ENV / CONFIG:** APP_ORIGIN; session/branding defaults.

**IMPLEMENT:** Create platform/client views for tenants, onboarding, leads, inbox, providers, knowledge, workflows, channels/numbers, usage, settings and audit. Use one Next.js/React application, accessible forms, responsive layout, loading/empty/error states and clear current-client identity. Every visible control must connect to authorized server data; no decorative fake metrics.

**SECURITY:** Server-side role checks; prevent data retained in client cache after tenant switch; credentials remain masked.

**TESTS:** Browser role tests, keyboard navigation, tenant switching and error/retry states; summaries match database fixtures.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/17.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Hide new navigation behind feature flags; preserve APIs and user preferences. **PRESERVE:** No separate per-client application or duplicate React SPA.

PRIMARY REFERENCES: S02

### PROMPT 18 | Implement versioned configuration and publishing

**OBJECTIVE:** Give dashboard changes safe draft, validation, publish and rollback. **PREREQUISITES:** Prompt 17 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** configuration module; config editor/diff/publish UI; schema registry. **DATABASE:** Config drafts, immutable published revisions, active revision pointer and audit.

**API CONTRACTS:** /configuration/drafts; validate; publish; rollback using expectedVersion. **ENV / CONFIG:** Only platform defaults in env; tenant settings live in MySQL.

**IMPLEMENT:** Define typed client profile, products, fields, rules, channels, AI, consent, scheduling, quotas and retention settings. Validate references and policy constraints before publish. Show a readable diff and impact preview; atomically change the active revision. Pin workflow runs to a revision while always rechecking current suppression/security.

**SECURITY:** Role-based editing/publishing; prevent configuration from changing immutable platform security boundaries.

**TESTS:** Invalid config cannot publish; concurrent publish conflicts; rollback restores prior config without reviving prohibited contact.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/18.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Publish a prior valid revision; audit the rollback and affected active runs. **PRESERVE:** No eval, arbitrary code, raw SQL or executable config.

# Prompts 19-20

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 19 | Create reusable client profiles and white-label settings

**OBJECTIVE:** Package repeatable business requirements as portable configuration. **PREREQUISITES:** Prompt 18 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** profile catalogue; tenant profile wizard; branding/locale settings UI. **DATABASE:** Profile versions, capability flags and tenant overrides.

**API CONTRACTS:** /profiles; /tenants/:id/profile with preview/diff. **ENV / CONFIG:** Default profile and approved capability allowlist.

**IMPLEMENT:** Define a generic enquiry profile plus explicitly synthetic vehicle/finance example profiles. Configure products, lead fields/statuses, qualification questions, languages, timezone, working hours, branding and provider settings. Copy configuration references safely; do not clone contacts, messages, credentials or consents. Profiles support upgrades with an override-preserving diff.

**SECURITY:** A profile cannot enable unapproved voice/marketing or grant permissions; branding assets are validated.

**TESTS:** Second tenant installs profile with no code edit; upgrade preserves overrides; secrets and customer data never clone.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/19.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Reapply prior profile/config revision; retain tenant-specific override history. **PRESERVE:** Profiles are data, not application forks.

### PROMPT 20 | Build dynamic forms and custom-field mapping

**OBJECTIVE:** Let admins change intake forms and field mappings safely. **PREREQUISITES:** Prompt 19 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** form builder; schema validator; lead-field renderer; import mapper. **DATABASE:** Field definitions, form versions and validated custom values.

**API CONTRACTS:** /forms; /field-mappings; form submission contract. **ENV / CONFIG:** Upload/form size limits; supported field-type registry.

**IMPLEMENT:** Support text, number, enum, date, boolean and validated contact fields with labels, required rules and conditional visibility. Map channel/provider fields to the lead schema through a preview. Preserve form version and original source reference. Validate on the server; accessibility and error messages match the configured schema.

**SECURITY:** Reject unsafe expressions/HTML; custom fields cannot overwrite tenant, roles, consent evidence or protected IDs.

**TESTS:** Malformed/missing values fail clearly; renamed fields keep stable IDs; mapping preview handles unknown fields; old records remain readable.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/20.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior form/mapping version; do not drop historic custom values. **PRESERVE:** Customizing a form must not require schema forks.

# Prompts 21-22

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 21 | Build the no-code routing rule editor

**OBJECTIVE:** Allow business users to configure deterministic routing. **PREREQUISITES:** Prompt 20 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** routing evaluator; rules editor; dry-run preview; assignment integration. **DATABASE:** Versioned routing conditions, priorities, capacity references and evaluation evidence.

**API CONTRACTS:** /routing/rules; /routing/evaluate; /routing/simulate. **ENV / CONFIG:** Approved operator/tie-breaker registry.

**IMPLEMENT:** Use typed conditions for product, geography and approved validated facts; configure priority, stable tie-breakers, capacity and manual-review fallback. Preview outcomes against synthetic or authorized sample leads. Persist reasons and rule version; the assignment transaction still enforces active-slot/capacity invariants. Block contradictory or unreachable rule sets when detected.

**SECURITY:** AI cannot select the final provider; editors cannot reference unauthorized provider/tenant IDs.

**TESTS:** Deterministic repeat evaluation, overlapping rules, exhausted capacity, concurrent assignment and no-match fallback tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/21.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Publish previous routing revision; reconcile affected assignments rather than silently reroute existing accepted work. **PRESERVE:** Assignment invariant, consent and provider scope.

### PROMPT 22 | Define the no-code workflow language

**OBJECTIVE:** Specify supported workflow nodes before writing a runtime. **PREREQUISITES:** Prompt 21 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** packages/engine DSL schemas; validator; docs/workflow-contract.md. **DATABASE:** Workflow definitions, revisions and immutable node identifiers.

**API CONTRACTS:** /workflows draft/validate contract; typed node registry. **ENV / CONFIG:** Maximum nodes/depth/runtime and allowed action registry.

**IMPLEMENT:** Define event trigger, condition, set validated field, routing request, wait, permitted message/call intent, AI qualification, handoff and end nodes. Specify input/output schemas, bounded branching, timeouts, revision pinning and error paths. Reject cycles unless explicitly bounded and supported; define duplicate-trigger keys. No arbitrary JavaScript/Python/SQL in client workflows.

**SECURITY:** Publishing cannot bypass policy or create role/consent changes; connector actions use restricted capabilities.

**TESTS:** Invalid edges, unbounded cycles, unknown actions, cross-tenant references and missing required terminal/error paths fail validation.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/22.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Keep DSL disabled until its compatibility and runtime contracts pass. **PRESERVE:** Versioned contract and mandatory safety boundaries.

# Prompts 23-24

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 23 | Implement durable workflow execution

**OBJECTIVE:** Execute published configuration reliably through the Node core. **PREREQUISITES:** Prompt 22 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** workflow runtime; worker integration; scheduler/reconciler; run-state API. **DATABASE:** Workflow runs, node attempts, durable waits, effects, receipts and revision references.

**API CONTRACTS:** /workflow-runs; pause/resume/cancel; internal trigger dispatch. **ENV / CONFIG:** WORKFLOW_CONCURRENCY; validated timeout/retry defaults.

**IMPLEMENT:** Persist each node transition/effect in MySQL and enqueue wake-ups through the outbox. Resume durable waits after restarts; deduplicate triggers and node effects. Run only supported nodes and pinned definitions. Cancellation blocks future effects, with final current consent/permission checks before every message/call. Track compensation/recovery where external effects cannot be undone.

**SECURITY:** Jobs must carry trusted tenant/service context; user config never executes raw code.

**TESTS:** Crash at every transition, duplicate trigger, cancelled wait and mixed revisions; no repeated external effect after reconciliation.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/23.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause runtime/new triggers; reconcile existing effects and resume on prior compatible engine. **PRESERVE:** No queue-only workflow state or silent revision switch.

### PROMPT 24 | Build workflow builder, simulator and publishing UI

**OBJECTIVE:** Expose the runtime through a usable visual no-code editor. **PREREQUISITES:** Prompt 23 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** apps/web workflow canvas/form editor; simulator; publish/revision history UI. **DATABASE:** Draft layout metadata, simulations and published definition references.

**API CONTRACTS:** /workflows/simulate; publish; revisions; runs. **ENV / CONFIG:** Simulator is sandboxed with fake outbound adapters.

**IMPLEMENT:** Support add/connect/edit nodes, condition forms, error-path warnings, preview and plain-language run trace. Provide a form/list alternative to a canvas. Simulations use synthetic or authorized copied inputs and cannot send messages/calls. Publish validated definitions, show differences and explain whether old runs remain pinned or are cancelled.

**SECURITY:** Separate editor/publisher rights; imported definitions pass server validation and cannot include secrets.

**TESTS:** Build a lead-to-route-to-permitted-follow-up workflow without code; invalid graph blocked; simulator has zero external effects.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/24.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior published revision; disable only future triggers while preserving in-flight evidence. **PRESERVE:** Simulator isolation and pinned active runs.

# Prompts 25-26

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 25 | Create connector registry and capability lifecycle

**OBJECTIVE:** Make supported integrations reusable per client. **PREREQUISITES:** Prompt 24 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** packages/adapters registry; connector module; dashboard connector catalogue. **DATABASE:** Connector definitions, tenant instances, asset bindings, health and external-approval states.

**API CONTRACTS:** /connectors; connect/test/disconnect; capability status. **ENV / CONFIG:** Platform app references; tenant credentials are secret-store references.

**IMPLEMENT:** Define adapter methods for authorization, discovery, verification, subscription, normalized inbound, permitted outbound, reconciliation and disconnect. Keep connected, permission_verified and active states distinct. Support multiple phone numbers/assets per tenant and immutable asset ownership bindings. Disconnect stops new effects and triggers recovery/offboarding. New vendors require a reviewed adapter; configuring a supported vendor requires no code.

**SECURITY:** Reject an asset bound to another tenant unless an approved transfer completes; do not trust browser-supplied asset ownership.

**TESTS:** Fake adapter tests discovery, revoke, duplicate asset binding and disconnect during queued delivery.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/25.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Deactivate connector instance; preserve event history and unresolved effects. **PRESERVE:** One scoped adapter interface, no arbitrary user endpoint execution.

### PROMPT 26 | Implement the communication policy and intent service

**OBJECTIVE:** Centralize authorization before any outbound action. **PREREQUISITES:** Prompt 25 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** communication-intent module; policy evaluator; final execution guard. **DATABASE:** Intents, permission decisions, attempt limits, status and provider correlation.

**API CONTRACTS:** /communication-intents; cancel; internal pre-send/pre-dial decision. **ENV / CONFIG:** Policy defaults and channel capability versions; deny on missing policy.

**IMPLEMENT:** Create intents for replies, templates, campaigns and calls; evaluate purpose, consent, current window, account status, recipient, limits, suppression and human ownership. Recheck immediately before adapter execution. Track pending/blocked/executing/accepted/uncertain/completed states. Cancel queued work on opt-out, tenant pause or revoked integration.

**SECURITY:** Policy is server-enforced and cannot be loosened by client prompt/config; uncertain external effects require reconciliation.

**TESTS:** Opt-out race, pause/revoke, prohibited template/window and ambiguous timeout tests; blocked intent never calls adapter.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/26.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable dispatch by channel/tenant; preserve intent decisions and reconcile accepted effects. **PRESERVE:** No direct send/dial from browser, AI or workflow.

PRIMARY REFERENCES: S10

# Prompts 27-28

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 27 | Create multi-client Meta readiness and review checklist

**OBJECTIVE:** Replace own-business assumptions with a verified customer-onboarding model. **PREREQUISITES:** Prompt 26 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Meta readiness wizard; docs/meta-evidence.md; connector prerequisites. **DATABASE:** Review evidence, app capability/access levels and per-client ownership records.

**API CONTRACTS:** /connectors/meta/readiness; status/evidence only, not fabricated Meta routes. **ENV / CONFIG:** META_APP_ID, META_APP_SECRET_REF, META_API_VERSION; verified setup configuration references.

**IMPLEMENT:** Browse current official Meta docs and account dashboard before coding any endpoint/scope. Verify the applicable Tech Provider/customer-onboarding path, app review, business verification and permissions. Record primary URLs, check date and blocked pages. Readable Meta Embedded Signup examples can include legacy flows; confirm current Cloud API contracts and billing eligibility. Display UI MAY DIFFER - VERIFY CURRENT META DASHBOARD where needed.

**SECURITY:** No copying an owner token to unrelated clients; no screen-scraping credentials or claiming review is approved.

**TESTS:** Readiness state accurately blocks missing approval; evidence distinguishes login/blocked from readable documentation; test actor is asset-authorized.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/27.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Keep connector pending_external; retain readiness evidence. **PRESERVE:** Do not invent permissions, button names, endpoints or approval timelines.

PRIMARY REFERENCES: S13 S14 S15

### PROMPT 28 | Implement secure customer authorization and Embedded Signup

**OBJECTIVE:** Let clients connect supported Meta assets through an official flow. **PREREQUISITES:** Prompt 27 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Meta onboarding adapter; authorized callback handler; dashboard connect action. **DATABASE:** Authorization attempts, state/nonces, verified assets and credential references.

**API CONTRACTS:** Own /connectors/meta/authorize and callback routes; external routes verified at implementation. **ENV / CONFIG:** META_LOGIN_CONFIG_ID when applicable; callback origin allowlist; app secret reference.

**IMPLEMENT:** Implement the current supported Embedded Signup/customer authorization flow after readiness evidence exists. Bind initiation to signed-in tenant admin, validate callback/state, origin, expiry and replay, and exchange codes server-side. Discover/check client-owned assets and required subscriptions; track partial success and billing/registration prerequisites. Use restricted assisted onboarding as a labelled fallback when eligible, not an unsafe token shortcut.

**SECURITY:** Never expose exchanged secrets; prevent callback tenant substitution, CSRF, code replay and asset theft.

**TESTS:** Success, cancelled popup, stale state, wrong origin, partial provisioning, repeated callback and revoked access tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/28.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disconnect/revoke newly authorized access; retain pending evidence for retry. **PRESERVE:** Current verified flow only; legacy collection examples are not production authority.

PRIMARY REFERENCES: S13 S14

# Prompts 29-30

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 29 | Build phone-number onboarding and safe replacement

**OBJECTIVE:** Let an admin add a verified number through the dashboard. **PREREQUISITES:** Prompt 28 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** number/assets UI; verification/provisioning state machine; connector adapter. **DATABASE:** Number asset IDs, ownership, registration status, capabilities, default sender and revision.

**API CONTRACTS:** /channel-assets; verify/test; select-default; deactivate. **ENV / CONFIG:** Number eligibility/registration settings from current verified provider contract.

**IMPLEMENT:** Support multiple numbers per client. Display registration/OTP/migration or coexistence prerequisites supplied by the provider; do not equate editing a phone string with integration. Test inbound/outbound permissions before selecting a sender. Replace the default for new intents while pinning old intents to their original asset, or explicitly cancel/recreate them with audit. Keep histories and client ownership intact.

**SECURITY:** Only authorized admins can connect/replace; redact OTPs; prevent reusing another tenant's number credentials.

**TESTS:** New supported number requires no code/deploy; failed verification stays inactive; old queued intents cannot silently switch sender.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/29.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior verified default; deactivate new asset and reconcile its effects. **PRESERVE:** Asset ownership, message history and explicit intent sender.

PRIMARY REFERENCES: S13 S14

### PROMPT 30 | Implement verified shared Meta webhook delivery

**OBJECTIVE:** Resolve every webhook to the correct connected client asset. **PREREQUISITES:** Prompt 29 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Meta webhook routes; raw-body verifier; asset resolver; inbox adapter. **DATABASE:** Use inbox records plus trusted connector/asset mapping.

**API CONTRACTS:** /webhooks/meta and configured aliases; verification challenge and delivery schemas. **ENV / CONFIG:** META_APP_SECRET_REF; META_VERIFY_TOKEN_REF; HTTPS public callback origin.

**IMPLEMENT:** Verify current challenge/signature contracts from primary sources. Preserve raw bytes for signature checks; validate app/asset, timestamp where applicable and supported payload shape. For multi-asset batches, create scoped durable records without cross-tenant mixing. Acknowledge only committed receipt; quarantine unknown assets/schema and handle re-delivery. Use webhook secret for the correct platform app, not a tenant-supplied fallback.

**SECURITY:** Reject invalid signatures; impose body limits; no sensitive payloads in unrestricted logs.

**TESTS:** Challenge verification, invalid signature, duplicate batch, unknown asset and two-client mixed-batch fixtures.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/30.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause affected asset; keep inbox evidence and recover provider retries after repair. **PRESERVE:** Trusted asset-to-tenant resolution and durable acknowledgement.

PRIMARY REFERENCES: S15 S16

# Prompts 31-32

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 31 | Implement Facebook Lead Ads retrieval adapter

**OBJECTIVE:** Retrieve real enquiry fields through verified client authorization. **PREREQUISITES:** Prompt 30 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Lead Ads adapter; retrieval worker; asset/form configuration UI. **DATABASE:** Source lead references, retrieval attempts and attribution metadata; business lead mapping follows next stage.

**API CONTRACTS:** Lead Ads notification/retrieval contracts verified from official sources; own /lead-sources status. **ENV / CONFIG:** META_API_VERSION; retrieval timeouts; scoped credential references.

**IMPLEMENT:** Confirm current permissions, Page/lead access and test-tool behaviour. Retrieve permitted fields after durable notification receipt; preserve campaign/ad/form/source IDs and raw field provenance. Handle pagination/contract changes only as required; transient retrieval failures remain pending, permanent access errors alert the owner. Never synthesize a full lead from a notification ID.

**SECURITY:** Use the credential authorized for that Page/tenant; no unrestricted fallback token or fake consent mapping.

**TESTS:** Official test lead retrieves expected fields; duplicate notification reuses source record; access failure is visible and safe.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/31.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable affected source subscription/worker; retain retrieval records for retry. **PRESERVE:** No guessed external routes or permissions.

PRIMARY REFERENCES: S17 S18

### PROMPT 32 | Implement normalization, qualification fields and golden routing

**OBJECTIVE:** Complete capture-to-provider across configured client requirements. **PREREQUISITES:** Prompt 31 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** normalizer; mapping UI integration; lead/routing workers; stage gate tests. **DATABASE:** Validated leads/source events, provenance and assignment/outbox transaction.

**API CONTRACTS:** /field-mappings preview; normalized event v1; routing request contract. **ENV / CONFIG:** Country/locale defaults from client config, not hardcoded tenant names.

**IMPLEMENT:** Map supported intake fields into the common lead schema; validate phone, product and location with original evidence preserved. Use approved field mappings and source dedup keys. Run deterministic routing, create one assignment or manual-review reason, and prepare permitted follow-up intents. Quarantine missing required fields. AI-extracted facts remain provisional until validation, not automatic underwriting.

**SECURITY:** Scope every write; do not infer marketing/voice permission from submission.

**TESTS:** Test lead -> durable inbox -> one enquiry -> eligible provider; replay/concurrency/unknown mapping and no-provider tests pass.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/32.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause normalization/routing; correct mapping version and replay safely with existing effect receipts. **PRESERVE:** Shared lead schema and active-assignment invariant.

# Prompts 33-34

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 33 | Implement WhatsApp outbound intents and status reconciliation

**OBJECTIVE:** Send only authorized replies/templates from each client's verified sender. **PREREQUISITES:** Prompt 32 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** WhatsApp adapter; communication worker; delivery status mapper. **DATABASE:** Outbound messages, attempt records, provider IDs, receipt transitions and errors.

**API CONTRACTS:** Own /messages intent API; current Cloud API sending/status contract verified before implementation. **ENV / CONFIG:** Scoped WhatsApp credential references; META_API_VERSION; bounded timeout policy.

**IMPLEMENT:** Select explicit tenant asset and permitted content. Use verified current API schemas; preserve provider IDs and map status/error evidence without regression. Distinguish queued, accepted and delivered. On timeout after a possible send, mark uncertain and reconcile before retry; where reconciliation cannot prove safety, require operator action. No blind repeat send.

**SECURITY:** Recheck current consent/window/suppression, asset access and human ownership immediately before sending.

**TESTS:** Authorized test recipient, invalid parameters, opt-out race, out-of-order status and uncertain timeout tests; no duplicate effect after replay.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/33.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Stop affected sender dispatch; reconcile in-flight records and restore prior verified adapter/config. **PRESERVE:** No direct adapter call bypassing intent policy.

PRIMARY REFERENCES: S10 S16

### PROMPT 34 | Implement WhatsApp inbound conversation processing

**OBJECTIVE:** Turn verified customer messages into a scoped conversation and task. **PREREQUISITES:** Prompt 33 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** WhatsApp inbound mapper; conversation worker; lead-linking flow. **DATABASE:** Messages, conversation links, current service-window evidence and inbound receipts.

**API CONTRACTS:** Normalized message.received event; conversation/task API. **ENV / CONFIG:** Allowed payload versions/media limits; tenant service-window policy references.

**IMPLEMENT:** Map text, supported interactive responses and status events separately. Track customer-origin message timestamps for current messaging eligibility. Create or continue the authorized conversation, preserve sender/asset identity, and trigger human/AI workflow only once. Customer enquiries may create/update a lead through validated tools; linking private existing enquiries requires sufficient identity evidence.

**SECURITY:** Retrieved/user text cannot authorize access; unknown sender identities cannot read private status.

**TESTS:** Repeat inbound delivery, two clients with similar numbers, human-owned thread, unsupported payload and private-status identity checks.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/34.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable automation trigger while retaining inbound capture for human review. **PRESERVE:** Verified identity scope and one message effect.

PRIMARY REFERENCES: S10 S16

# Prompts 35-36

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 35 | Build WhatsApp template, media and account-health management

**OBJECTIVE:** Let clients manage approved messaging assets from the dashboard. **PREREQUISITES:** Prompt 34 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** template catalogue/editor; media adapter; health/status UI. **DATABASE:** Template versions/language/category/status; media references; quality/limit evidence.

**API CONTRACTS:** /templates; submit/sync/test; /media authorized access; /connector-health. **ENV / CONFIG:** Private object-store references; current provider size/type limits.

**IMPLEMENT:** Support current verified template management and approval lifecycle, parameter validation and sender-language selection. Sync rather than assume approval. Handle restricted media retrieval, scanning and lifecycle. Show current account access/quality/limits where exposed. For outside-window contact, enforce applicable approved templates and policy; confirm effective rates instead of hardcoding free messaging.

**SECURITY:** No sensitive finance/identity values in routine templates; private media uses short-lived scoped access.

**TESTS:** Unapproved template cannot send; changed parameters fail; blocked account pauses sends; cross-tenant media and malware rejected.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/35.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior template/config selection; suspend unsafe media/message capability. **PRESERVE:** Approval and quality are external state, not editable success flags.

PRIMARY REFERENCES: S10 S16 S19

### PROMPT 36 | Implement Instagram through its verified login path

**OBJECTIVE:** Reuse shared conversations with the correct Instagram-specific authorization. **PREREQUISITES:** Prompt 35 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Instagram adapter; connector configuration; inbound/outbound mappers. **DATABASE:** Professional account asset bindings, scoped identities and delivery evidence.

**API CONTRACTS:** Current official Instagram login/messaging/webhook contract; own connector actions. **ENV / CONFIG:** Platform app reference and scoped client credential metadata.

**IMPLEMENT:** Verify the selected current login path, professional-account eligibility, scopes and webhook setup from official docs. Do not mix permissions from other Instagram integration paths. Map messages to the unified conversation model, use communication intents for replies and preserve current messaging restrictions and provider IDs. Unknown capabilities remain disabled with a clear readiness reason.

**SECURITY:** Account ownership, tenant filters and policy enforced; no unverified cross-channel merge or scraped DM automation.

**TESTS:** Authorized two-account fixtures, valid/invalid webhooks, repeated messages, revoked access and prohibited outbound behaviour.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/36.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Deactivate Instagram automation; retain inbound/audit history and reconcile pending sends. **PRESERVE:** Shared lead/conversation contracts and verified platform restrictions.

PRIMARY REFERENCES: S20 S21

# Prompts 37-38

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 37 | Implement Messenger with scoped Page access

**OBJECTIVE:** Add Messenger using the existing adapter and policy interfaces. **PREREQUISITES:** Prompt 36 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Messenger adapter; Page connector UI; inbound/outbound worker mapping. **DATABASE:** Page asset bindings, Page-scoped identities, messages and status evidence.

**API CONTRACTS:** Current official Messenger Page messaging/webhook contract; own connector actions. **ENV / CONFIG:** Platform app/secret references; scoped Page-authorized credential references.

**IMPLEMENT:** Verify current Page permissions/access levels, sender token actor, subscriptions and messaging restrictions. Reuse verified inbox, normalized conversation and communication-intent flows. Handle supported message/event types and provider error/status records. Do not treat generic Page administration as blanket outreach permission. Keep unsupported payloads visible for operator triage.

**SECURITY:** Page identity scope includes tenant/asset; same user identifiers from different scopes cannot leak private enquiry data.

**TESTS:** Two Pages/tenants, repeat delivery, invalid signature, revoked scope and policy-blocked send tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/37.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable Page messaging dispatch; retain durable events for controlled replay. **PRESERVE:** Common domain/events and policy-before-send.

PRIMARY REFERENCES: S22

### PROMPT 38 | Add website forms and controlled contact import

**OBJECTIVE:** Support first-party intake without hardcoded client-specific endpoints. **PREREQUISITES:** Prompt 37 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** form embed/hosted UI; import preview worker; ingestion adapters. **DATABASE:** Import manifests, row errors, source IDs and validated enquiries.

**API CONTRACTS:** /forms/:publicId/submit; /imports preview/commit/status with declared schemas. **ENV / CONFIG:** Approved public form origin rules; upload limits and import-size policy.

**IMPLEMENT:** Generate client forms from published schemas; bind public form IDs to server-resolved tenants. Add spam/rate controls, source attribution and approved notice text. CSV import previews mappings/errors and separates contact data from permission evidence. Bulk commit uses idempotent row keys and bounded jobs. Label sample records synthetic; imported rows cannot trigger campaigns by default.

**SECURITY:** Validate uploads/CSRF as appropriate; prevent formula/export injection; imported consent needs evidence, not a checkbox assumption.

**TESTS:** Malformed row, duplicate import, wrong form scope, replay and spam checks; preview creates no outbound effects.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/38.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Cancel pending import; undo only newly created safe records through audited actions. **PRESERVE:** No implicit communication consent or arbitrary schema writes.

# Prompts 39-40

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 39 | Build the human inbox and operator handoff

**OBJECTIVE:** Give people a reliable way to take over customer conversations. **PREREQUISITES:** Prompt 38 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** inbox dashboard; conversation timeline; handoff/assignment UI; SSE service. **DATABASE:** Handoff states, operator assignments, notes and read cursors.

**API CONTRACTS:** /handoffs; /conversations ownership; authorized event stream. **ENV / CONFIG:** SSE connection/time limits; no tenant data in shared HTTP cache.

**IMPLEMENT:** Provide unified channels, filters, enquiry context, internal notes, response controls and unresolved-error banners. Human takeover atomically pauses AI automation; resumption is explicit. Reuse communication intents for operator replies and display delivery/blocked status. SSE is tenant-authorized and reconnects safely; late AI responses must not send after takeover.

**SECURITY:** Customer/provider roles see only allowed messages; notes and phone values have field-level restrictions.

**TESTS:** Human/AI race, disconnected browser, removed operator access and tenant-switch stream tests; operator reply still obeys policy.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/39.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable live updates or auto-resume; fall back to authorized polling and manual ownership. **PRESERVE:** Human ownership and communication checks are server state.

### PROMPT 40 | Build lead-management and provider workspaces

**OBJECTIVE:** Deliver the internal LMS as a lead-management dashboard. **PREREQUISITES:** Prompt 39 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** lead board/detail/filter UI; provider portal; operational reporting APIs. **DATABASE:** Saved views, follow-up tasks, status history and provider response records.

**API CONTRACTS:** /lead-views; /tasks; assignment accept/reject; scoped list/detail APIs. **ENV / CONFIG:** Tenant-configured statuses and fields; upload/export size limits.

**IMPLEMENT:** Build list/board views with search, product/source/provider filters, configured fields, task scheduling and complete lead timeline. Provider portal accepts/rejects assignments and records permitted updates. All metrics come from durable records; saved views and custom statuses follow published config. Provide an internal lead-management workspace alongside client and provider dashboards.

**SECURITY:** Provider can access only authorized assignments/fields; bulk exports are permissioned and audited.

**TESTS:** No-code field/status changes appear correctly; parallel provider actions respect versions; lists, totals and filters match fixtures.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/40.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior published view/status config; preserve lead/provider history. **PRESERVE:** No client-specific code, fictitious metrics or unsafe bulk edits.

# Prompts 41-42

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 41 | Implement the private Python AI service boundary

**OBJECTIVE:** Use Python for computation without a second business-data authority. **PREREQUISITES:** Prompt 40 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** services/ai-service; generated contracts; Node AI client and durable job integration. **DATABASE:** AI jobs/runs and outcome references in MySQL through Node; Python has no business write access.

**API CONTRACTS:** Private /internal/v1/embed, retrieve and generate contracts with deadlines/request IDs. **ENV / CONFIG:** AI_SERVICE_URL, AI_SERVICE_AUTH_REF, approved model-provider secret references.

**IMPLEMENT:** Add FastAPI validation, bounded concurrency/deadlines, cancellation and verified model/embedding adapters. Node supplies an authorized context bundle and records results. Python returns validated proposals/evidence, not direct customer sends or database writes. Use durable Node jobs for long work; apply model capability limits rather than unsupported universal parameters. Pin prompt/model/embedding versions per run.

**SECURITY:** Authenticate services, prevent public exposure, redact data and reject tenant/context substitution; no MySQL or channel credentials in Python.

**TESTS:** Schema parity, timeout/cancel, duplicate job and unauthorized service call tests; no direct send/write path exists.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/41.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable AI capability and use approved human fallback; retain job evidence. **PRESERVE:** Node owns business writes and all effects.

PRIMARY REFERENCES: S03

### PROMPT 42 | Build knowledge upload, review and lifecycle

**OBJECTIVE:** Let each client manage an approved knowledge library. **PREREQUISITES:** Prompt 41 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** knowledge UI/module; parse jobs; private upload/scanning adapter. **DATABASE:** Documents, versions, approvals, chunks metadata and indexing jobs.

**API CONTRACTS:** /knowledge/documents; approve/publish/archive; indexing status. **ENV / CONFIG:** OBJECT_STORE_* references; approved file types/size; parser limits.

**IMPLEMENT:** Support document/text uploads and narrowly controlled source ingestion. Parse with size/time budgets, retain source/page provenance, preview chunks and require approval before retrieval. Version content, scope product/locale and handle replacement/deletion. Indexing is durable and resumable; publication does not claim success until derived index readiness is proven.

**SECURITY:** Private storage; SSRF-safe allowlisted ingestion; uploaded text is data and cannot alter system policy.

**TESTS:** Bad file, parser failure, duplicate upload, approval gate and replacement/deletion tests; draft content cannot answer questions.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/42.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Unpublish version; restore prior approved version and reconcile its index. **PRESERVE:** Approved knowledge is distinct from private chat memory.

# Prompts 43-44

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 43 | Implement Qdrant collections and index consistency

**OBJECTIVE:** Add scalable vector search with mandatory tenant and access filtering. **PREREQUISITES:** Prompt 42 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Qdrant adapter; indexing worker; reconciliation/deletion jobs; collection schemas. **DATABASE:** MySQL remains document/memory authority; record vector point/version/job references.

**API CONTRACTS:** Private index/query/delete adapter; no direct browser Qdrant API. **ENV / CONFIG:** QDRANT_URL, QDRANT_API_KEY_REF; embedding dimension/version and collection names.

**IMPLEMENT:** Use separate collections by data purpose/schema, not automatically one collection per small client. Index tenant/product/locale/access/version metadata. Every query includes trusted tenant/ACL filters; shard routing does not replace them. Upsert deterministic point IDs, track tombstones and reconcile delayed writes so deletion cannot be undone by an old indexing job. Use verified Qdrant features supported by the pinned version.

**SECURITY:** Private network/TLS/restricted keys; authorize before search and validate returned ownership/version against current authority.

**TESTS:** Identical text across two tenants stays isolated; repeated upsert is stable; delete-versus-index race cannot resurrect content.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/43.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause indexing; rebuild derived collections from authorized MySQL records and active versions. **PRESERVE:** No vector store as primary business database.

PRIMARY REFERENCES: S04 S05 S23

### PROMPT 44 | Implement scoped RAG retrieval and evidence

**OBJECTIVE:** Return relevant approved passages with explainable provenance. **PREREQUISITES:** Prompt 43 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Python retrieval pipeline; Node authorization/context service; retrieval test corpus. **DATABASE:** Retrieval evidence/run references; no raw private context in public cache.

**API CONTRACTS:** Private retrieval request filters/token budget and evidence response schema. **ENV / CONFIG:** Allowed embedding/reranker/model settings; bounded retrieval budgets.

**IMPLEMENT:** Filter approved active content by tenant, product, locale and actor access before search. Use measured dense/sparse or keyword retrieval and reranking only when justified; deduplicate and enforce context budgets. Validate source/version availability before returning citations. Missing evidence produces uncertainty/handoff rather than guessed financing or policy answers. Keep conversation retrieval behind separate private authorization.

**SECURITY:** Retrieved documents are untrusted data; strip instruction influence and prohibit source content from authorizing tools.

**TESTS:** Two-tenant leakage, stale document, injection passage, multilingual query and unsupported answer tests using labelled synthetic fixtures.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/44.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Switch to approved simpler retrieval or human fallback; retain provenance and versions. **PRESERVE:** Current authorized sources, not similarity alone.

PRIMARY REFERENCES: S04 S23

# Prompts 45-46

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 45 | Implement structured memory, summaries and archive

**OBJECTIVE:** Remember useful facts without carrying all history into the model. **PREREQUISITES:** Prompt 44 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** memory/summary modules; extractor jobs; context assembler; history lifecycle UI. **DATABASE:** Memory items, provenance/confidence/expiry, summaries, raw-history/archive references.

**API CONTRACTS:** /memory authorized read/correct/delete; private extraction/summary schemas. **ENV / CONFIG:** Context-token budget; approved memory sensitivity/retention policy.

**IMPLEMENT:** Separate recent task context, verified facts/preferences, episodic events, procedural knowledge and conversation summaries. Extract candidate facts with source references; validate and apply deterministic correction/version rules. Use current lead records over stale memories, retrieve only relevant authorized items and expire obsolete facts. Raw history stays restricted with approved lifecycle; users can correct or remove derived memory.

**SECURITY:** No identity/finance secrets in reusable memory; isolate per tenant/person/conversation and reject injected instructions as facts/policy.

**TESTS:** Conflicting/corrected fact, deleted summary/vector/cache, similar-name identity and context-budget tests; current status wins over old memory.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/45.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable memory retrieval/extraction; use recent authorized context without deleting source evidence. **PRESERVE:** No cross-person semantic memory or unsupported fact promotion.

### PROMPT 46 | Implement public semantic cache using Redis and Qdrant

**OBJECTIVE:** Reuse safe answers without creating a private-data leak. **PREREQUISITES:** Prompt 45 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** semantic-cache adapter; invalidation jobs; cache policy and metrics. **DATABASE:** Public answer metadata/source versions; private Redis keys remain scoped temporary context.

**API CONTRACTS:** Cache lookup requires exact scope/version filters before similarity; private queries return bypass. **ENV / CONFIG:** REDIS_URL; public-cache TTL/budget/threshold config; approved cache collection.

**IMPLEMENT:** Use Redis for answer payload/expiry and Qdrant for optional public-answer similarity candidates. Exact filters include tenant/product/locale, approved knowledge version, policy and prompt/model/embedding versions. Validate candidate freshness before reuse. Do not add a second Redis vector engine by default. Cache hits cannot skip current authorization; record invalidation on correction, publication and deletion.

**SECURITY:** Never cache private lead status, transcripts, personal memory, identity or finance answers in a shared semantic collection.

**TESTS:** Same question different tenant/version, expired Redis response, false similarity, deleted knowledge and private query tests; unsafe requests always bypass.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/46.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable semantic lookup; keep exact safe caching or uncached retrieval with bounded cost. **PRESERVE:** Private answers never become shared public cache.

# Prompts 47-48

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 47 | Create authorized AI tools and runtime prompt composition

**OBJECTIVE:** Give the agent useful capabilities through typed server-enforced tools. **PREREQUISITES:** Prompt 46 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** tool registry; policy/context assembler; prompt-version editor; audit integration. **DATABASE:** Prompt revisions, tool executions, authorized proposals and AI-run evidence.

**API CONTRACTS:** Tools: searchKnowledge, readLead, captureFacts, createLead, requestHandoff, proposeReplyIntent. **ENV / CONFIG:** Approved provider/model allowlist and service credential references.

**IMPLEMENT:** Compose platform policy, channel/business policy, role/task, authorized context, retrieval/memory/tool data and response schema. Provide separate extractor, scorer, summarizer, answerer, caller and handoff instructions. Client-editable prompt sections cannot weaken immutable security/policy. Node validates tools and performs writes using current authorization, versions/idempotency. AI may suggest relevance/qualification scores with evidence, never final provider or credit approval.

**SECURITY:** Reject injected tool calls, raw SQL, role/consent edits and arbitrary HTTP; Python/model has no privileged credentials.

**TESTS:** Schema-invalid tool, injection, stale update, forged tenant and provider-choice attempt tests; all rejected with audit.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/47.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable affected tool/prompt revision and restore prior approved version. **PRESERVE:** Deterministic policy and routing remain outside AI.

### PROMPT 48 | Implement omnichannel AI chat and human fallback

**OBJECTIVE:** Complete useful chat over the shared channel engine. **PREREQUISITES:** Prompt 47 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** chat orchestrator; channel response formatter; AI settings/sandbox UI. **DATABASE:** AI runs, reply proposals/intents, qualification facts and handoff records.

**API CONTRACTS:** POST /ai/chat async; status; preview/test using fake outbound adapters. **ENV / CONFIG:** Approved language/model/context limits and per-tenant AI budgets.

**IMPLEMENT:** Select the published client agent configuration, current channel rules and authorized context. Answer approved product/process questions, capture requirements, read authorized status and request human help. Enforce evidence, finance/eligibility boundaries and human ownership; late responses are suppressed after takeover/withdrawal. Format within verified channel capabilities and send only through intents. Provide client sandbox before activation.

**SECURITY:** No unsupported promises or autonomous consent/provider decisions; redact sensitive facts and enforce tenant quotas.

**TESTS:** Multi-turn continuity, no-answer handoff, injection, private status, language settings, late model response and opt-out races.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/48.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause AI replies; route incoming messages to humans; restore prior prompt/config revision. **PRESERVE:** All channels share one lead/memory/policy authority.

# Prompts 49-50

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 49 | Create India voice readiness and compliance workflow

**OBJECTIVE:** Keep calling off until the actual client/route is approved. **PREREQUISITES:** Prompt 48 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** voice readiness UI; provider evidence checklist; compliance policy records. **DATABASE:** Per-client purpose/route/number/disclosure/recording approvals and launch decisions.

**API CONTRACTS:** /voice/readiness; evidence submission; activation check. **ENV / CONFIG:** Calling flag defaults false; approved route/purpose policy references.

**IMPLEMENT:** Review current TRAI/India requirements and provider guidance for the actual business purpose and launch date. Verify commercial communication, consent, suppression/DND, number series and automation requirements with the provider/legal owner. Separate WhatsApp permission from voice/recording permission. Record legal evidence, expiry/review dates and ownership. The proposed 09:00-19:00 window is a business default, not a legal claim.

**SECURITY:** No UI switch can self-certify legal/provider approval; recording off until separately authorized.

**TESTS:** Missing/expired evidence, suppression and withdrawn consent block activation; readiness status shows accountable blocker/owner.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/49.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable calling and recording; preserve approval/audit evidence. **PRESERVE:** No guaranteed legal clearance or activation time.

PRIMARY REFERENCES: S11 S24 S25

### PROMPT 50 | Implement Exotel voice connectivity adapter

**OBJECTIVE:** Connect approved India routes through a reusable provider interface. **PREREQUISITES:** Prompt 49 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** voice adapter; connectivity health/test UI; reconciliation client. **DATABASE:** Voice intents/attempts, route/number references and provider call IDs.

**API CONTRACTS:** Verified Exotel SIP/connectivity/dial/status contracts; own /voice/test intent. **ENV / CONFIG:** VOICE_PROVIDER_SECRET_REF, approved trunk/route references, bounded call timeouts.

**IMPLEMENT:** Confirm the supported Exotel/managed-agent route and current verified APIs. Configure tenant route/number references in the dashboard, authenticate callbacks and preserve correlation. Separate attempted, provider-accepted, connected and completed states. Apply final pre-dial policy and attempt budget. A timeout after possible initiation becomes uncertain; reconcile before dialing again. Do not assume global telephony eligibility from a generic voice API.

**SECURITY:** Restricted provider credentials, approved recipients for tests and private SIP credentials; no direct dial outside intent execution.

**TESTS:** Approved test route, refusal/throttling, duplicate callback, opt-out race and uncertain dial timeout tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/50.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable dial worker for the route; reconcile ongoing calls and restore prior verified settings. **PRESERVE:** India readiness gate and no blind redial.

PRIMARY REFERENCES: S24 S25

# Prompts 51-52

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 51 | Configure managed AI voice agents and bounded tools

**OBJECTIVE:** Make voice behaviour configurable without rebuilding the application. **PREREQUISITES:** Prompt 50 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** ElevenAgents integration; voice-profile UI; prompt/tool sandbox. **DATABASE:** Agent/profile revisions, language/question configuration and tool audit references.

**API CONTRACTS:** Verified agent/SIP/tool contracts; restricted signed server tool callback. **ENV / CONFIG:** Managed voice service credential reference; approved language/voice/transfer configuration.

**IMPLEMENT:** Verify current ElevenAgents SIP and tool interfaces. Configure approved greeting/identity disclosure, qualification questions, language, interruption/error handling, duration and human transfer. Client edits create a draft requiring preview/publish. Tools validate signed call context and can only capture permitted facts/read authorized lead data/request handoff. Agent cannot determine credit, choose provider, grant consent or initiate arbitrary follow-up.

**SECURITY:** No secrets in spoken prompts/tool payloads; recording and transcript permissions are independently checked.

**TESTS:** Spoofed tool/call ID, language mismatch, unsupported financial answer, prompt injection and failed human transfer tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/51.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior approved agent revision; pause new calls while active sessions finish/reconcile safely. **PRESERVE:** Current call permissions and deterministic business rules.

PRIMARY REFERENCES: S26 S27

### PROMPT 52 | Implement post-call outcomes and callback reconciliation

**OBJECTIVE:** Translate verified call evidence into safe lead updates. **PREREQUISITES:** Prompt 51 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** voice callback verifier; outcome mapper; transcript lifecycle; reconciliation job. **DATABASE:** Calls, attempts, structured outcomes, transcript/media refs and outcome receipts.

**API CONTRACTS:** Verified provider post-call/status contracts; own /voice-calls authorized reads. **ENV / CONFIG:** Callback signing-secret references; outcome retention and size limits.

**IMPLEMENT:** Validate callbacks and source IDs; match tenant/call/lead through trusted stored references. Preserve raw provider outcome versus model summary separately. Apply idempotent outcomes, non-regressing status and bounded structured qualification extraction. Reconcile callbacks from connectivity and managed-agent providers without double-counting the call. Restricted transcripts/recordings follow approved consent/retention.

**SECURITY:** Reject unauthenticated or mismatched callbacks; no transcript embedding before privacy policy permits it.

**TESTS:** Duplicate/out-of-order/partial callbacks, missing transcript, failed call and two-provider completion races; one business outcome.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/52.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause outcome writes; retain evidence for reviewed reprocessing with receipts. **PRESERVE:** Call evidence cannot fabricate consent or customer statements.

PRIMARY REFERENCES: S28

# Prompts 53-54

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 53 | Implement qualification and explainable lead scoring

**OBJECTIVE:** Summarize facts without automating underwriting or final assignment. **PREREQUISITES:** Prompt 52 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** qualification/scoring module; score explanation UI; configurable rubric. **DATABASE:** Validated qualification answers, rubric versions, score evidence and reviewer corrections.

**API CONTRACTS:** /qualifications; /scores; preview/simulate with authorized inputs. **ENV / CONFIG:** Approved rubric weights/thresholds in published tenant config.

**IMPLEMENT:** Map chat/call facts to product, location, timeframe and approved needs. Separate missing, uncertain, model-inferred and customer-confirmed values. Compute a configurable deterministic relevance score with reasons; AI may propose facts or summaries, validated before use. Allow authorized correction and rerun. Routing uses approved rules, not an opaque score that silently overrides them.

**SECURITY:** Do not infer protected traits or promise financing/credit eligibility; protect sensitive fields and record provenance.

**TESTS:** Missing/inconsistent facts, rubric revision, correction and noisy transcript tests; score is reproducible and never performs credit approval.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/53.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore previous rubric; retain original score/version and audit corrections. **PRESERVE:** Provider/credit decision boundaries and evidence provenance.

### PROMPT 54 | Build scheduling, campaigns and approved follow-up automation

**OBJECTIVE:** Let clients configure bounded follow-up through workflows. **PREREQUISITES:** Prompt 53 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** schedule/campaign UI; durable scheduler; communication/policy integration. **DATABASE:** Campaign drafts, recipient snapshots, schedule revisions, intent/effect receipts and cancellations.

**API CONTRACTS:** /campaigns preview/publish/pause; /schedules; recipient eligibility preview. **ENV / CONFIG:** Per-tenant timezone/quiet hours/attempt limits and channel quotas.

**IMPLEMENT:** Create follow-up schedules with explicit purpose, channel, approved content and recipient criteria. Preview counts and suppression reasons; schedule through durable workflow waits/outbox, not browser timers. Apply current permission/window/quality limits to every recipient at execution. Pause immediately stops new intents, while in-flight sends reconcile. No unrestricted bulk cold-contact feature.

**SECURITY:** Permissioned campaign publishing, bounded costs/rates and no consent inferred from imported contact lists.

**TESTS:** Timezone/DST boundaries, opt-out during wait, tenant pause, budget limit and duplicate campaign execution tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/54.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause campaign/scheduler; cancel unsent intents and reconcile accepted effects. **PRESERVE:** No automation bypasses final contact permission.

PRIMARY REFERENCES: S10 S11

# Prompts 55-56

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 55 | Add reusable outbound integrations and secure mappings

**OBJECTIVE:** Support approved client CRM/provider destinations without arbitrary code. **PREREQUISITES:** Prompt 54 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** connector action registry; field-mapping UI; signed outbound webhook adapter. **DATABASE:** Destination configs, credential refs, delivery attempts and receipts.

**API CONTRACTS:** /destinations configure/test; typed connector actions; status/replay. **ENV / CONFIG:** Server-enforced destination allowlist/network restrictions and secret references.

**IMPLEMENT:** Provide reviewed generic webhook/CRM adapter actions with configurable field mapping and declared payload schema. Validate destination ownership, TLS, authentication and allowed data fields. Sign outbound payloads where supported; retain delivery correlation and bounded retries. A provider action can notify an assignment but cannot bypass its state/capacity checks. New connector types need reviewed code; new supported destinations use dashboard config.

**SECURITY:** SSRF/DNS-rebinding protection, private-network deny rules and no secrets/PII beyond authorized destination purpose.

**TESTS:** Forbidden URL/IP, redirect attack, invalid mapping, duplicate delivery and revoked credential tests; safe destination config needs no deploy.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/55.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Deactivate destination; reconcile delivery attempts and restore prior validated config. **PRESERVE:** No arbitrary HTTP nodes, raw code or unrestricted export.

### PROMPT 56 | Create delivery packages and guided client onboarding

**OBJECTIVE:** Turn the supported system into a repeatable service package. **PREREQUISITES:** Prompt 55 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** onboarding wizard; package catalogue; readiness dashboard; test-run guide. **DATABASE:** Package/profile versions, client requirement questionnaire, checklist evidence and activation state.

**API CONTRACTS:** /delivery-packages; install/preview; /onboarding/checklist; activate when ready. **ENV / CONFIG:** Approved package capability registry; credentials supplied through vault/connect flows.

**IMPLEMENT:** Guide the operator through business requirements, products/fields, rules/providers, numbers/channels, knowledge, consent, AI/voice, schedules, branding and quotas. Import a reusable configuration package without secrets or personal data. Run synthetic simulation then approved live tests. Show internal setup time separately from Meta/provider approvals and client-content delays. One-to-three-day delivery is a target for ready supported clients, not a guaranteed platform approval time.

**SECURITY:** Client activation requires required evidence and authorized approval; do not clone another client's data/assets.

**TESTS:** A new synthetic client installs a package with no code edits; missing access blocks only affected capabilities; partial onboarding resumes safely.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/56.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause/deactivate package capabilities; restore prior config, preserving evidence. **PRESERVE:** No template can bypass permissions, isolation or external review.

# Prompts 57-58

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 57 | Implement data classification, retention and deletion

**OBJECTIVE:** Delete private source and derived data across every store. **PREREQUISITES:** Prompt 56 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** privacy module/UI; lifecycle jobs; deletion reconciler and restore ledger. **DATABASE:** Classification policies, deletion requests/store receipts, retention revisions and restricted tombstones.

**API CONTRACTS:** /privacy/requests; export/correct/delete/status with authorization. **ENV / CONFIG:** Approved retention-policy refs; private archive/backup lifecycle configuration.

**IMPLEMENT:** Classify contact data, enquiries, messages, transcripts, recordings, consent, prompts/memory, embeddings and logs. Require business/legal-approved retention; recommendations are not statutory defaults. Erase or restrict records as authorized, remove Qdrant/Redis/summary/media derivatives, and prevent delayed jobs from resurrecting them. Preserve only approved minimal evidence and replay deletion ledger after restore.

**SECURITY:** Identity verification, restricted request access and no privacy export containing another tenant or secrets.

**TESTS:** Delete during embedding/send job, repeated erasure, partial store outage and backup restore tests; forbidden derivatives cannot resurface.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/57.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause faulty deletion job, preserve tombstones and retry approved erasure; never undo withdrawal/deletion casually. **PRESERVE:** Deletion covers all stores and restores.

PRIMARY REFERENCES: S29

### PROMPT 58 | Harden the dashboard, APIs, uploads and integrations

**OBJECTIVE:** Close security gaps before serving external clients. **PREREQUISITES:** Prompt 57 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** security middleware; threat model; upload/connector validators; dependency policy. **DATABASE:** Security incident/evidence metadata; no unplanned schema rewrite.

**API CONTRACTS:** Enforce auth/CSRF/rate/body limits consistently on all exposed routes. **ENV / CONFIG:** TLS/CORS/session/security-header settings; trusted proxy configuration.

**IMPLEMENT:** Review IDOR, tenant injection, unsafe redirects, SSRF, callback replay, uploads, XSS, secret leaks, SQL injection and configuration/tool abuse. Use allowlisted schemas, output encoding and scoped server authorization; restrict administrative and service networks. Validate trusted proxies and callback URLs, pin supported dependencies and document patch responsibility. Focus on actual attack paths; avoid decorative security claims.

**SECURITY:** Least privilege for MySQL/Qdrant/Redis/storage and service principals; support grants auditable and revocable.

**TESTS:** Adversarial two-tenant tests, malformed input, URL-rebinding, stored XSS, token/log scan and revoked-session/tool tests.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/58.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable vulnerable capability and revoke exposed credentials; preserve incident evidence and apply compatible patch. **PRESERVE:** No security weakening to pass a failing test.

# Prompts 59-60

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 59 | Implement per-client usage, quotas and budget limits

**OBJECTIVE:** Make shared capacity and billable usage accountable. **PREREQUISITES:** Prompt 58 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** usage meter; quota evaluator; admin/client usage UI; reconciliation jobs. **DATABASE:** Usage ledger, reservation/settlement records, plan limits and provider bill references.

**API CONTRACTS:** /usage; /quotas; reserve/settle/reconcile service contracts. **ENV / CONFIG:** Approved provider/model rates with effective date; no invented tariff constants.

**IMPLEMENT:** Meter leads, AI tokens, embeddings, messages, connected call duration, storage and workflow execution. Reserve/check budgets before expensive work, settle actual evidence and handle retry/idempotency. Distinguish estimated versus measured charges and platform fees versus provider costs. Use tenant fair scheduling and alert thresholds; client plan edits cannot increase privileged limits. Online payments are optional later, not required for usage visibility.

**SECURITY:** Restrict billing/rate edits; redact customer content; one tenant cannot exhaust another's quota.

**TESTS:** Concurrent reservations, duplicate callback, failed attempt, rate revision and budget exhaustion tests; totals reconcile to evidence.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/59.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause affected expensive capability; reconcile reservations without deleting usage history. **PRESERVE:** Never advertise simulated costs as current provider prices.

PRIMARY REFERENCES: S19

### PROMPT 60 | Build operational and client analytics

**OBJECTIVE:** Show trustworthy performance and funnel information. **PREREQUISITES:** Prompt 59 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** analytics queries/aggregates; dashboard charts; report/export UI. **DATABASE:** Versioned metric definitions, aggregate jobs and scoped saved reports.

**API CONTRACTS:** /analytics funnel/channel/provider/usage with bounded ranges and filters. **ENV / CONFIG:** Aggregation intervals and authorized reporting dimensions.

**IMPLEMENT:** Define lead capture, qualification, assignment, acceptance, follow-up, handoff, delivery and call-outcome metrics precisely. Distinguish accepted send from delivered message and attempted dial from connected call. Explain missing/late data and model-derived categories. Use MySQL indexes/aggregates before introducing a separate analytics stack. Provide per-client reporting and aggregate platform health without revealing customer content.

**SECURITY:** Tenant/role filters on every query/export; privacy thresholds where aggregation could expose a person.

**TESTS:** Known synthetic timelines yield expected counts; duplicates do not inflate metrics; late events reconcile; cross-tenant/report-range limits enforced.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/60.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Revert new aggregate version and rerun from durable events; keep original evidence. **PRESERVE:** No fake revenue, conversion, latency or savings metrics.

# Prompts 61-62

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 61 | Implement monitoring, alerts and recovery console

**OBJECTIVE:** Give an operator actionable visibility across clients. **PREREQUISITES:** Prompt 60 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** observability middleware; worker metrics; alerts; recovery/dead-letter UI. **DATABASE:** Incident/recovery actions, audit and durable failure reasons.

**API CONTRACTS:** /operations queues/events/intents; authorized reconcile/replay, never unrestricted resend. **ENV / CONFIG:** OTEL_EXPORTER_*; alert destinations configured without sending test messages to others.

**IMPLEMENT:** Trace correlation across webhook, MySQL/outbox, workers, Python and providers. Monitor queue age, stuck workflows, retrieval/access failures, uncertain effects, quota and deletion backlog. Every alert names tenant/capability, owner and safe next action. Recovery UI previews replay impact and checks current policy/effect receipts. Choose pilot thresholds from measured traffic, not fabricated SLO achievements.

**SECURITY:** Restrict operator/support scope; log references instead of private transcripts/tokens; audit every replay.

**TESTS:** Outage alert, stuck lease, noisy tenant, uncertain send and unauthorized replay tests; trace explains one full enquiry journey.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/61.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable faulty alert or replay action; use documented manual reconciliation with audit. **PRESERVE:** No blind resend/redial button or private content in shared monitoring.

### PROMPT 62 | Package a portable deployment and infrastructure configuration

**OBJECTIVE:** Run the same product on shared SaaS or a dedicated client deployment. **PREREQUISITES:** Prompt 61 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** deploy containers/IaC templates; CI deployment; runbooks; network topology. **DATABASE:** Compatible migration job and durable storage configuration; no tenant-by-tenant schema drift.

**API CONTRACTS:** Readiness/liveness and private service topology; public API/browser routing. **ENV / CONFIG:** Global DB/Redis/Qdrant/storage/auth/service-secret references; per-client assets stay in dashboard/vault.

**IMPLEMENT:** Build reproducible images and deploy API, worker, Next.js and Python with MySQL, Redis, Qdrant and private storage. Start with a managed or small container deployment; Kubernetes/Kafka are not required for the initial team. Keep databases and AI endpoints private; specify TLS, backups, resource limits and separate staging. Publish reader-compatible contracts before new writers and run approved migrations safely.

**SECURITY:** Separate deployment/runtime/migration privileges; no secrets in images or client export packages.

**TESTS:** Clean staging deploy, health failure, compatible upgrade, worker drain and rollback rehearsal.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/62.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Roll back application images/flags compatibly; use approved forward data fix instead of destructive migration reversal. **PRESERVE:** Shared codebase and versioned deployment contract.

# Prompts 63-64

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 63 | Implement backups, restore and disaster-recovery drill

**OBJECTIVE:** Prove recoverability across durable and derived stores. **PREREQUISITES:** Prompt 62 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** backup/restore runbooks; restore scripts; reconciliation/deletion ledger integration. **DATABASE:** MySQL backup/restore state, object manifests and restricted deletion ledger.

**API CONTRACTS:** Operator-only restore/reconciliation procedures; no public destructive route. **ENV / CONFIG:** Backup encryption/key refs, retention and approved recovery objectives.

**IMPLEMENT:** Back up MySQL and private object data with tested consistency. Treat Qdrant as reconstructible but document snapshots/rebuild and model/version requirements. Redis can be recreated from durable work. Restore to an isolated environment first, replay deletion/suppression ledgers, reconcile intents and verify credentials/ownership before any sends/dials. Measure achieved recovery times; do not claim zero data loss without evidence.

**SECURITY:** Encrypted restricted backups; restore does not grant old revoked users access or resurrect deleted data.

**TESTS:** Restore a synthetic tenant, cancelled intent and deleted memory; no external effects until release gate; consistency checks pass.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/63.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Keep production unchanged; discard failed isolated restore and repair procedure. **PRESERVE:** Recovery never blindly replays external effects.

PRIMARY REFERENCES: S05

### PROMPT 64 | Benchmark and plan MySQL/Qdrant growth

**OBJECTIVE:** Scale based on measurements without redesigning every client. **PREREQUISITES:** Prompt 63 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** load harness; query/index tuning; Qdrant capacity/deployment ADR; quota tests. **DATABASE:** Measured MySQL indexes/aggregates; vector placement metadata only if necessary.

**API CONTRACTS:** Preserve public contracts; version internal search/deployment adapters. **ENV / CONFIG:** Measured concurrency/replication/resource settings; no invented throughput targets.

**IMPLEMENT:** Generate labelled synthetic volumes for enquiries/messages/vectors and test representative mixes. Measure latency, memory, queue age, recall and isolation under noisy neighbours. Tune MySQL pools/indexes/archiving and scale workers independently. Evaluate Qdrant distribution/replication and tiered tenant placement only on supported versions and measured need; retain mandatory tenant filters. Dedicated client deployments use the same release/configuration format.

**SECURITY:** Capacity tests use no real customer data or messages; dedicated shards are not substitutes for authorization.

**TESTS:** Publish hardware/dataset/config and raw results; verify large-tenant load cannot leak or starve smaller tenants; reindex/deletion remains correct.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/64.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore previous measured settings/index plan; reshard only through documented compatible provider procedures. **PRESERVE:** No universal most-advanced or guaranteed million-scale claim.

PRIMARY REFERENCES: S04 S05

# Prompts 65-66

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 65 | Create AI evaluation and regression gates

**OBJECTIVE:** Measure answer/tool/qualification quality before expanding automation. **PREREQUISITES:** Prompt 64 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** AI eval corpus/harness; review dashboard; prompt/model release gates. **DATABASE:** Evaluation runs, labelled expected outcomes and reviewer decisions.

**API CONTRACTS:** /ai/evaluations authorized job/status; never a production messaging path. **ENV / CONFIG:** Approved model/retrieval/prompt versions and bounded test budget.

**IMPLEMENT:** Build explicitly synthetic/authorized cases for supported questions, financing boundaries, unsupported facts, multilingual chat/voice, memory corrections, injection and cross-tenant leakage. Measure evidence correctness, tool authorization, handoff and qualification accuracy with transparent rubric. Compare prompt/model/config changes; set approved pilot thresholds and review failures. No claim of perfect answers or benchmark performance without measured evidence.

**SECURITY:** Private evaluations isolated; adversarial content cannot trigger real tools, calls or exports.

**TESTS:** Known failing prompt is blocked; corrected/deleted knowledge affects results; unauthorized-tool and cross-tenant cases always fail safe.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/65.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Restore prior approved prompt/model/retrieval version; disable affected automated task. **PRESERVE:** No test corpus disguised as real production success.

### PROMPT 66 | Run official channel contract and account acceptance tests

**OBJECTIVE:** Prove actual supported account access for each enabled channel. **PREREQUISITES:** Prompt 65 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** channel contract fixtures; account readiness evidence; dashboard acceptance checklist. **DATABASE:** Redacted test evidence, asset IDs, provider IDs and receipts.

**API CONTRACTS:** Verified current Meta/provider contracts; own APIs remain unchanged. **ENV / CONFIG:** Isolated test sender/recipient and authorized asset/credential refs.

**IMPLEMENT:** Refresh official API/policy evidence before activation. Exercise Lead Ads retrieval, WhatsApp inbound/outbound/template/status, Instagram, Messenger and approved voice as configured. Check app/token/asset scope, webhook verification/signature and current messaging restrictions. Mark blocked approvals as pending_external, not passing tests. Record current dashboard differences without inventing labels. External tests use approved recipients only.

**SECURITY:** No production customer contact or account setting changes beyond authorized test scope; redact credentials.

**TESTS:** Expected event, database change, policy decision and provider status match for every enabled connector; disabled/pending capability cannot execute.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/66.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable failed connector capability; retain evidence and reconcile test effects safely. **PRESERVE:** A configured connector is not a verified production integration.

PRIMARY REFERENCES: S10 S13 S16 S17 S20 S22 S24

# Prompts 67-68

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 67 | Run failure, race and load verification

**OBJECTIVE:** Prove recovery and isolation at the platform boundaries. **PREREQUISITES:** Prompt 66 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** integration/chaos/load suites; test environment; recovery reports. **DATABASE:** Synthetic event/run/intent fixtures with durable receipts and state versions.

**API CONTRACTS:** Test all existing contracts under duplicate, delay, timeout and revoked-access scenarios. **ENV / CONFIG:** Isolated test network and fake provider/model adapters; measured run settings.

**IMPLEMENT:** Inject MySQL/Redis/Qdrant/model/provider outages, crash windows, queue loss, callback reordering, concurrent assignment, deletion/index races and quota contention. Reconcile uncertain effects and delayed outbox jobs. Validate aggregate ordering and backward-compatible payloads. Load shared tenants with differing traffic. Record actual results/resources and prioritize fixes before advancing; do not rename failed tests as expected success.

**SECURITY:** No chaos on production or real sends/dials; preserve scoped secrets and redacted evidence.

**TESTS:** Durable work recovers, prohibited contact stays blocked and no duplicated external effect is blindly retried; all required race tests pass.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/67.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Stop fault injection and restore isolated snapshot/config; verify production remains unchanged. **PRESERVE:** Tenant security and effect receipts cannot be removed for speed.

### PROMPT 68 | Verify complete business journeys and dashboard usability

**OBJECTIVE:** Prove that the system meets the original three-system requirements. **PREREQUISITES:** Prompt 67 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Playwright journeys; integration scenarios; operator training walkthrough. **DATABASE:** Synthetic tenants/leads/conversations, live evidence only where approved.

**API CONTRACTS:** All published user/operator/provider APIs and connector actions. **ENV / CONFIG:** Published tenant profiles and approved isolated test credentials.

**IMPLEMENT:** Run form/Lead Ads -> normalized lead -> deterministic assignment -> provider response; WhatsApp -> approved RAG -> scoped memory -> handoff; approved call -> verified outcome -> qualification. Exercise configuration publish/rollback, new number, profile change, opt-out and privacy deletion. A nondeveloper should complete supported client setup from the dashboard using the guide. Fix broken navigation and confusing blockers.

**SECURITY:** Use minimum necessary data; sensitive/private statuses require verified identity; all outbound tests authorized.

**TESTS:** Every journey matches durable records and trace; unsupported questions hand off; operator can explain and recover failed work without code.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/68.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause failed capability and restore prior config; keep business/audit evidence. **PRESERVE:** No frontend-only simulation counted as an integrated pass.

# Prompts 69-70

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 69 | Perform release security, privacy and compliance review

**OBJECTIVE:** Review every activated capability against actual evidence. **PREREQUISITES:** Prompt 68 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** release checklist; threat model updates; compliance decision records. **DATABASE:** Approval/evidence references, expiry and accountable owners.

**API CONTRACTS:** No new contract unless explicitly approved; inspect all exposed surfaces. **ENV / CONFIG:** Validate secret references, production flags and policy versions without exposing values.

**IMPLEMENT:** Review authentication/MFA, tenant scope, provider/customer roles, webhooks, tools, SSRF, logs, retention/deletion, restore, quotas and incident response. Have business/provider/legal owners confirm applicable Meta/India/purpose/recording requirements. Verify published prompt/config cannot override safety. Classify remaining issues and keep unsupported routes disabled. Do not infer legal approval from passing technical tests.

**SECURITY:** Privilege and live-traffic access require the correct recorded owners; remediate real findings before activation.

**TESTS:** Two-tenant adversarial regression, secret scan, opt-out race, deletion restore and approved connector readiness evidence reviewed.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/69.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Keep release pending; disable unsafe capability, rotate compromised credentials and patch compatibly. **PRESERVE:** No self-certified legal clearance or hidden release blocker.

PRIMARY REFERENCES: S10 S11 S23 S29

### PROMPT 70 | Implement tenant export, transfer and dedicated import

**OBJECTIVE:** Make a client portable without moving other clients or platform secrets. **PREREQUISITES:** Prompt 69 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** tenant export/import UI/jobs; manifest validator; dedicated deployment runbook. **DATABASE:** Scoped export/import manifests, ID remap, approval and transfer audit.

**API CONTRACTS:** /tenant-transfers preview/export/import/validate; asynchronous restricted operations. **ENV / CONFIG:** Destination trust/encryption refs; per-client external authorization re-established.

**IMPLEMENT:** Export approved configuration and, separately with authority, client data/media. Include schema/release versions, checksums, referenced IDs, consent/suppression/deletion evidence and active-effect status. Exclude platform credentials/other tenants. Import remaps IDs and rebuilds derived indexes; require the recipient to reconnect authorized external assets and secrets. Freeze or explicitly hand off active workflows/calls before cutover to avoid two systems contacting the same customer.

**SECURITY:** Encrypted time-limited exports, least-privilege approvals and proof of client/asset ownership; never reassign WABA by editing a DB field.

**TESTS:** Export contains only one tenant; compatible import preserves evidence; missing credentials stay inactive; duplicate import creates no duplicate effects.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/70.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Abort before cutover; restore original active deployment and reconcile handoff effects. **PRESERVE:** Transfer is ownership-aware, versioned and auditable.

# Prompts 71-72

Apply Prompt 00 and the standard preflight/postflight. Proposed paths map to existing equivalents.

### PROMPT 71 | Deploy the approved pilot with rollback evidence

**OBJECTIVE:** Activate a small controlled release, not every unfinished feature. **PREREQUISITES:** Prompt 70 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** release pipeline; feature flags; pilot runbook; go/no-go evidence. **DATABASE:** Approved production migration/evidence; no destructive schema reversal.

**API CONTRACTS:** Enable only verified API/connector/workflow capabilities; maintain compatible readers. **ENV / CONFIG:** Production secret refs; tenant/channel flags; actual deployment/monitoring settings.

**IMPLEMENT:** Select a ready supported client, run staging/restore tests and migrations, publish approved config, then enable only reviewed capabilities. Schedule operators and provider escalation; monitor capture, assignment, messages, uncertain calls and deletion/usage. Rollback disables new effects, drains/reconciles in-flight work and returns to compatible images/config. Ten-day MVP is a planning target; the full no-code platform may take longer.

**SECURITY:** Required owner approvals and client permissions recorded; do not deploy with universal admin credentials or enabled unapproved voice.

**TESTS:** Pilot journeys, health, alert response, opt-out and rollback rehearsal pass; actual outcomes documented with known limitations.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/71.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Disable affected flags, reconcile effects and deploy previous compatible release; approved forward data repair if needed. **PRESERVE:** No release labelled complete while required evidence is missing.

### PROMPT 72 | Onboard a second client with zero code changes

**OBJECTIVE:** Prove the reusable no-code delivery promise and prepare handover. **PREREQUISITES:** Prompt 71 evidence reviewed; run the standard preflight. Also verify the dependencies named below.

**FILES:** Dashboard onboarding flow; delivery package; operator manual; final evidence report. **DATABASE:** A second unrelated tenant with its own config/assets/data and provisioning receipts.

**API CONTRACTS:** Use existing tenant/profile/config/connect/test/publish APIs only; no new hardcoded routes. **ENV / CONFIG:** Client credentials via approved dashboard/vault flow; no tenant-specific .env or redeploy.

**IMPLEMENT:** Use different branding, product fields, languages/timezone, routing/providers, knowledge, workflows and a new supported number. Configure all through the dashboard, simulate then run approved acceptance tests. Confirm client one remains isolated and unchanged. Measure internal onboarding time separately from pending external approvals. Produce operator/developer ownership guide, maintenance/update policy, known supported boundaries and portable package. Fix any step requiring code for supported configuration before declaring the platform reusable.

**SECURITY:** Second client cannot see first client's leads, memory, metrics, media or credentials; activation still respects policy gates.

**TESTS:** No source/config-file edits or rebuild required; all enabled journeys work; isolation, transfer and rollback evidence pass; unapproved features remain clearly pending.

**DELIVER:** Return changed files, migrations/contracts, actual test results, operator steps and docs/stages/72.md; record blockers. **DONE:** Required checks pass with evidence; configuration and recovery work. Unapproved external capabilities stay disabled.

**ROLLBACK:** Pause second client, reconcile its effects and retain evidence; do not disturb client one. **PRESERVE:** No promise of arbitrary integrations, perfect AI or guaranteed third-party approval.

# Official sources: 1 of 3

Checked 9 October 2026. Content read means accessible evidence; login/blocked/dynamic/legacy references require further account-specific verification. Official sources support choices, not an assurance of approval or performance.

**S01 MySQL 8.4: composite foreign-key constraints**
https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html
Evidence: Content read, 9 Oct 2026.

**S02 Next.js: Server and Client Components**
https://nextjs.org/docs/app/getting-started/server-and-client-components
Evidence: Content read, 9 Oct 2026.

**S03 FastAPI: deployment workers**
https://fastapi.tiangolo.com/deployment/server-workers/
Evidence: Content read, 9 Oct 2026.

**S04 Qdrant: multitenancy and tiered placement**
https://qdrant.tech/documentation/manage-data/multitenancy/
Evidence: Content read, 9 Oct 2026.

**S05 Qdrant: distributed deployment**
https://qdrant.tech/documentation/scaling/distributed_deployment/
Evidence: Content read, 9 Oct 2026.

**S06 Node.js: official release schedule**
https://github.com/nodejs/Release
Evidence: Primary indexed content read, 9 Oct 2026.

**S07 Fastify: validation and serialization**
https://fastify.dev/docs/latest/Reference/Validation-and-Serialization/
Evidence: Content read, 9 Oct 2026.

**S08 MySQL 8.4: locking reads**
https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html
Evidence: Content read, 9 Oct 2026.

**S09 Prisma: MySQL support (current redirect target)**
https://www.prisma.io/docs/orm/v7/core-concepts/supported-databases/mysql
Evidence: Old /orm/overview/databases/mysql redirected here; content read.

**S10 Business Messaging Policy**
https://whatsappbusiness.com/policy/
Evidence: Redirect verified; content read.

# Official sources: 2 of 3

### S11 Regulations register

https://trai.gov.in/release-publication/regulations

Evidence: Content read.

### S12 Production guide

https://docs.bullmq.io/guide/going-to-production

Evidence: Content read.

### S13 Meta-owned Embedded Signup collection

https://www.postman.com/meta/whatsapp-business-platform/documentation/du6gzjv/embedded-signup?entity=request-13382743-3252619f-be88-4fe4-8da3-851038d1e2de

Evidence: Content read; contains legacy patterns - verify current Cloud API flow.

### S14 Meta Embedded Signup overview

https://developers.facebook.com/documentation/business-messaging/whatsapp/embedded-signup/overview

Evidence: Inaccessible during latest check; current flow must be verified.

### S15 Apps directory

https://developers.facebook.com/apps/

Evidence: Login redirect.

### S16 Official WhatsApp API collection

https://www.postman.com/meta/whatsapp-business-platform/documentation/wl

Evidence: Content read.

### S17 Lead retrieval guide

https://developers.facebook.com/docs/marketing-api/guides/lead-ads/retrieving/

Evidence: Blocked 429.

### S18 Lead retrieval setup checker

https://github.com/facebookincubator/catalogue-of-api-solutions/blob/main/solutions/leads/leads-retrieval-set-up-checker.md

Evidence: Content read.

### S19 Platform pricing

https://whatsappbusiness.com/products/platform-pricing/

Evidence: Redirect verified; content read.

### S20 Instagram platform

https://developers.facebook.com/docs/instagram-platform/

Evidence: Blocked 429.

# Official sources: 3 of 3

### S21 Instagram API official collection

https://www.postman.com/meta/instagram/documentation/6yqw8pt/instagram-api?entity=request-23987686-894be833-d0b6-4877-859e-c61ae6474d64

Evidence: Latest nested view dynamic/blank; primary indexed content read earlier.

### S22 Messenger collection

https://www.postman.com/meta/messenger-platform-api/documentation/iyp204x/messenger-platform-api

Evidence: Content read.

### S23 Qdrant: security and access control

https://qdrant.tech/documentation/security/

Evidence: Content read, 9 Oct 2026.

### S24 AgentStream selection guide

https://developer.exotel.com/docs/agentstream/what-to-use-when

Evidence: Content read.

### S25 Voice AI SIP trunk integration

https://docs.exotel.com/exotel-agentstream/exotel-sip-trunk-api-reference-for-voice-ai

Evidence: Indexed content read.

### S26 SIP trunking

https://elevenlabs.io/docs/eleven-agents/phone-numbers/sip-trunking

Evidence: Content read.

### S27 Webhook tools

https://elevenlabs.io/docs/eleven-agents/customization/tools/webhook-tools

Evidence: Content read.

### S28 Post-call webhooks

https://elevenlabs.io/docs/eleven-agents/workflows/post-call-webhooks

Evidence: Content read.

### S29 DPDP Act 2023

https://www.meity.gov.in/static/uploads/2024/02/Digital-Personal-Data-Protection-Act-2023.pdf

Evidence: Content read.